



**Universitatea POLITEHNICA din București
FACULTATEA DE INGINERIE MEDICALĂ**



PROIECT DE DIPLOMĂ

Clasificarea Hemangioamelor Infantile Utilizând Rețele Neuronale de tip Deep Neural Network

Student: Nicolas IGNĂTESCU

Conducător Științific: Conf. Dr. Ing. Alina SULTANA

**București
<Iulie 2018>**



Cuprins

CAPITOLUL 1: INTRODUCERE	2
1.1. JUSTIFICARE MEDICALĂ	4
1.2. STADIUL ACTUAL AL PROBLEMEI MEDICALE	6
1.3. PROBELE IMAGISTICE FOLOSITE ÎN PRACTICA MEDICALĂ ACTUALĂ	8
1.3.1. Stadiul actual al sistemelor pentru diagnoza asistată de calculator (cad) a leziunilor	9
1.3.2. Stadiul actual al sistemelor pentru evaluarea asistată de calculator a hemangioamelor infantile	10
1.4. REȚELE NEURONALE CONVOLUȚIONALE, DEEP NEURAL NETWORK	11
1.4.1. Starea actuală a cadrelor și librăriilor pentru „Machine Learning”	14
CAPITOLUL 2: DEZVOLTAREA REȚELEI NEURONALE	20
2.1. ARHITECTURA UNUI CNN ÎN MATLAB	20
2.2. PREPROCESAREA DATELOR	27
CAPITOLUL 3: ARHITECTURA CNN PROPUȘĂ ȘI IMPLEMENTAREA ACESTEIA	30
3.1. CNN ÎN DOUĂ CLASE	32
3.2. CNN ÎN TREI CLASE	34
CAPITOLUL 4: INTERPRETAREA REZULTATELOR	36
4.1. REZULTATE	36
4.1.1. Clasificarea HI în „Hem” și „NonHem”	37
4.1.1.1. Sub-imagini Color	37
4.1.1.2. Sub-imagini în spațiul G	38
4.1.2. Clasificarea HI după sesiunile de consultare	41
4.1.2.1. Sub-imagini color	41
4.1.2.2. Sub-imagini în spațiul G	43
4.1.3. Clasificarea a unei imagini reale	44
4.2. INTERPRETAREA REZULTATELOR	48
CAPITOLUL 5: DEZVOLTĂRI ULTERIOARE ȘI CONCLUZII	54
5.1. ÎNBUNĂTĂȚIRI VIITOARE	54
5.2. CONCLUZII	56
BIBLIOGRAFIE	57



CAPITOLUL 1: INTRODUCERE

În ultima vreme algoritmi de învățare automată au început să se dezvolte din ce în ce mai mult. Algoritmii **Deep Learning** (DL) stau la baza majorității aplicațiilor din momentul actual, de la camerele foto din telefoanele inteligente care îți îmbunătățesc pozele instantaneu până la inteligență artificială. Recunoașterea imaginilor de către algoritmi antrenați prin Deep Learning, în unele scenarii, pot fi mai buni decât oamenii. Domeniul medical poate folosi acești algoritmi pentru a crește eficiența serviciilor medicale. Alături de celelalte sisteme din spital DL poate oferi medicilor rapid informații importante pentru a-i ajuta în procesul de diagnosticare a unor patologii.

Conceptul de inteligență artificială a apărut în jurul anilor '50 când un grup de oameni de știință au dat naștere termenului la o conferință în Dartmouth, New Hampshire, USA. Ideea de **Artificial Intelligence** (AI) a fost însă considerată nerealizabilă datorită puterii de calcul semnificativ mai slabă decât cea din ziua de azi. Pe zi ce trece, tehnologia a avansat permițând cercetătorilor să folosească AI pentru a crea mașini care pot învăța să efectueze sarcini predeterminate. O altă abordare a algoritmilor de învățare sunt rețele neuronale. Acestea sunt inspirate din natură și din înțelegerea noastră a funcționării creierului. Sunt formate din neuroni care comunică între ei. Spre deosebire de sistemul biologic, unde neuronii pot avea conexiuni cu oricare dintre vecinii săi, neuronii unei rețele neuronale au conexiuni și straturi (sau layer) discrete. O astfel de rețea preia un semnal de intrare și îl desface în bucăți. Fiecare neuron primește datele de la stratul precedent, își execută sarcina cu acel semnal, și trimite rezultatul la stratul următor. Și tot așa până la ultimul strat care va da rezultatul analizei. Fiecare neuron atribuie ponderi datelor de intrare iar la sfârșit acesta ia o decizie în funcție de acestea. În procesarea de imagini cele mai folosite rețele neuronale sunt cele convoluționale



(Convolutional Neural Network sau CNN). CNN-urile s-au dovedit a fi eficiente în a clasifica imagini. Ideea de rețea neurală este destul de veche, însă acestea nu au dat rezultatele scontate din motive legate de dezvoltările tehnologiei și a puterii de calcul înregistrate la momentul respectiv. Asta până când, în 2012, Andrew Ng a propus creșterea numărului de date de intrare și a straturilor. „Deep” (tr. „Profund”) se referă la numărul de straturi dintr-o rețea neurală. Astăzi DL poate identifica patologii și clasifica imagini medicale cu o mare acuratețe. Un astfel de CNN a fost pus în aplicare și în această lucrare pentru a clasifica o imagine care conține un **Hemangiom Infantil (HI)** în diferite stadii ale acestuia.

În studiul HI este realizată pe scară largă fotografierea leziunilor pentru a monitoriza dezvoltarea acestora. În general, acestea sunt făcute cu o cameră comercială de tip DSLR, iar imaginile nu sunt de calitate. Fotografiile sunt foarte importante deoarece o schimbare a caracteristicilor patologiei poate avea complicații grave. Hemangioamele cutanate sunt cele mai comune tumori vasculare benigne care apar în copilărie. Aceste pot prolifera rapid și pot obstrucționa funcții vitale cum ar fi respirația, văzul, auzul, înghițitul sau excreția. Leziuni la nivelul feței sau gâtului pot provoca probleme psihologice viitoare copiilor sau părinților. Aria afectată sau reducerea acesteia în timp este dificil de măsurat și necesită mult din timpul medicului pentru ca acesta să compare imaginile precedente cu cele actuale. Din acest motiv, un sistem automat de detecție a evoluției hemangiomului care să ajute medicul să ia o decizie mai rapidă este de dorit. DL poate ajuta în acest aspect deoarece poate clasifica imaginile în mai multe categorii.

Hemangiomul infantil apare la naștere doar în proporție de 30% de obicei apare pe parcursul primului an de viață. Astfel incidența la nou născut crește de la 1,1 – 2.6% până la 12% [1]. Această afecțiune este cel mai des întâlnită la sexul feminin și ca localizare sunt de obicei în zona capului, gâtului și trunchiului [2]. HI este o patologie care, în general, nu prezintă complicații și dispare după un timp. În aceste cazuri nu este necesar un tratament special. Însă, la HI cu probleme (care au ulceratii, sângerări sau care au o dezvoltare rapidă) trebuie aplicate metode terapeutice. Medicii tratează această afecțiune prin: 1. Oprirea etapei de proliferare 2. Inhibarea accelerată a hemangioamelor importante 3. Prevenirea și tratarea problemelor funcționale. În principiu, etapa de proliferare a hemangiomului trebuie oprită cât mai repede.



Imaginile folosite în acest studiu au fost preluate de la Spitalul Clinic de Urgență pentru copii "Maria Sklodowska Curie" din București. Baza de date este formată din imagini care au ca obiect de interes HI. Imaginile provin de la mai mulți pacienți. Acestea sunt clasificate în sesiuni de consultație. Acestea reprezintă momentul în care pacientul s-a prezentat la un consult. Pacienții care au venit la consultații consecutive, sesiunile au ordin crescător (Sesiunea 1, Sesiunea 2 etc). Imaginile au fost realizate cu ajutorul unor camere foto digitale obișnuite și nu într-un mod standard. Din baza de date s-au ales 36 de imagini care s-au considerat folositoare pentru antrenarea rețelelor neuronale.

DL are nevoie de multe imagini pentru a fi eficient. Numărul de imagini poate ajunge și la un milion. Însă, dacă nu avem acces la o astfel de bază de date, putem reduce imaginile la sub-imagini. Adică, au fost alese matrice de câte 55x55 de pixeli din imaginea originală care au fost împărțite în categorii: Piele, Hemangiom sesiunea 1, Hemangiom sesiunea 2, Hemangiom sesiunea 3. Tipul de antrenare a CNN-ului este supervizat, adică la un număr de iterații rețeaua se va verifica singură cu un set de imagini care nu au fost folosite în învățare. Astfel putem urmări în timp real dacă rețeaua face overfitting. În general o astfel de învățare este mai încheată decât una nesupervizată dar, urmărind ultimele dezvoltări în materie de GPU în curând puterea de procesare, implicit și timpul necesar antrenării, nu o să mai prezinte o problemă.

În Capitolul 1 se va prezenta justificarea medicală, detalii despre HI, sumarul articolelor care prezintă dezvoltarea actuală în acest domeniu. Capitolul 2 cuprinde principiile DL, arhitectura și tipul rețelelor neuronale, precum și preprocesarea datelor. În continuare, Capitolul 3, va fi prezentată soluția proprie, implementarea propriu-zisă a rețelelor în MatLab și antrenarea acestora, precum și observațiile mele. În Capitolul 4 sunt prezentate rezultatele și interpretarea acestora. Capitolul 5 care va cuprinde concepte pentru dezvoltări ulterioare și concluzii.

1.1. JUSTIFICARE MEDICALĂ

Hemangioamele infantile sunt cele mai comune tumori vasculare benigne care apar în primii ani de viață. Aspectul clinic al HI depinde de stratul pielii în care apare. Hemangioamele superficiale au un aspect de „căpșună” în schimb cele de la niveluri mai adânci au un aspect de tumoare albăstruie. Un alt tip de HI este cel mix, acesta implică și straturile de piele superficiale



și adânci. Procentul în care acestea contribuie precum și alte caracteristici morfologice afectează dezvoltarea afecțiunii pe termen lung [3]. Un HI se poate dezvolta pe o întreagă regiune anatomică pe când unele se pot dezvolta în locuri discrete [4]. Mai mult de jumătate din HI apar pe cap și gât, favorizând anumite părți ale corpului, probabil datorită modului în care embrionul se dezvoltă [5]. Dezvoltarea HI are o fază de proliferare și una de recidivă [6]. Locația anatomică are o importanță majoră deoarece în faza de proliferare acestea pot obstrucționa funcții vitale cum ar fi respirația văzul, auzul înghițitul sau excreția.

De obicei HI intră în faza de recidivă de la sine, însă pentru cazurile în care nu se întâmplă acest lucru, medicul trebuie să decidă asupra unui tratament. În prezent opțiunile sunt variate și depinde de dotările clinicii. Posibile tratamente includ: excizia chirurgicală, tratament cu medicamente β - blocante, laser terapie și injecții de bleomicină.

În România cea mai folosită metodă terapeutică este excizia chirurgicală. La nivel internațional aceasta este recomandată numai după epuizarea celorlalte metode [7]. Succesul operației precum și aspectul estetic depind foarte mult de experiența medicului, locul dar și momentul în dezvoltarea HI în care s-a intervenit.

O mare parte din clinicile internaționale au introdus tratamentul farmacologic ca metodă principală de tratament al HI. Dr. Christine Leaute-Labreze de la departamentul de dermatologie pediatrică din cadrul spitalului de copii din Bordeaux, Franța a descoperit accidental că propranolol, care era folosit pentru a trata afecțiuni ale inimii, a inhibat dezvoltarea HI la doi copii. După o cercetare mai amănunțită rezultatele au fost publicate în jurnalul „New England Journal of Medicine” pe 12 Iunie 2008 [8]. În prezent încă nu se cunoaște metoda de acționare a medicamentului asupra dezvoltării HI. La noi în țară s-a introdus acest tratament în 2010.

Terapia cu fascicul Laser are rezultate foarte bune după 1-5 ședințe dar necesită ca pacientul să fie anesteziat, ceea ce prezintă un dezavantaj. Modul de funcționare a acestei metode constă în producerea unei inflamații la nivelul HI ceea ce obturează vasele sangvine din acesta. Astfel se pornește un proces de involuare rapidă a HI [1].



În cazul unor anumite tipuri de hemangioame se folosește tratamentul cu bleomicină. Acesta este o substanță sclerozantă care se injectează. Nu necesită anestezie generală și nu prezintă costuri ridicate de aceea este un tratament accesibil. În plus reacțiile adverse sunt minime.

Hemangiomul fiind benign, adică nu prezintă un risc iminent pentru sănătatea pacientului, medicii mai recomandă urmărirea pe un timp mai îndelungat. Astfel, în funcție de cum se dezvoltă condiția, se poate lua o decizie în ceea ce privește tratamentul.

În diagnosticarea unei astfel de afecțiuni trebuie ca diferența dintre HI și malformațiile vasculare să se facă rapid și eficient. În prezent un astfel de diagnostic se realizează dificil. Medicii nu pot recomanda un tratament până când nu există un diagnostic cert. Această lucrare își propune dezvoltarea unui algoritm de învățare de tip DL care va stabili o bază pentru standardizarea diagnosticului cât și pentru alegerea unui tratament optim. Leziunile lăsate netratate pot provoca sechele funcționale și estetice. Algoritmul va permite medicilor de familie, neonatologilor pediatrii, sau chirurgilor pediatrii să ajute la stabilirea diagnosticului medical. Radiologiile și tratamentele suplimentare, care nu sunt neapărat necesare, pot traumatiza și copii și părinții. Din acest motiv un astfel de ajutor din partea unui calculator este benefic.

1.2. STADIUL ACTUAL AL PROBLEMEI MEDICALE

Până în anii '80 nu se făcea distincția dintre tumori și malformații vasculare. În nomenclatură acestea erau descrise ca „semne vasculare din naștere” [7]. HI este puțin studiat la momentul actual iar un protocol bine definit pentru a urmări în timp sau a trata această afecțiune nu există.

Prima clasificare prin diferențierea hemangioamelor de malformații a fost realizată în 1982. Aceasta a fost determinată de observarea unei faze de proliferare urmată de una de involuție a tumorilor vasculare. În acest timp a fost înființată și Societatea Internațională de Studii a Anomaliilor Vasculare (ISSVA) care, în 1996, a înlocuit termenul de hemangiom cu cel de tumoră vasculară [9]. Totodată recomandă clasificarea acestora în superficiale profunde și mixte [3] [9]. În paralel s-a propus și o clasificare bazată pe aspectul clinic și gradul de afectarea a țesuturilor: localizate, segmentare, nedeterminate și multifocale.

Stadiu	Clinică	CCDS
1.Prodromal	Pete albe/roșiatice, teleangiectazii	Nu prezintă o structură specifică;
2.Inițial		Hipervascularizație la margine
3.Proliferativ	Colorit roșu intens al leziunii, piele indurată, reliefată, marginile infiltrate, creștere rapidă	Hiperperfuzie sangvină crescută difuz în cadrul tumorii; densitate crescută a vaselor sangvine
4.Maturație	Culoarea devine roșu deschis sau purpuriu; posibile ulceratii centrale; scade în dimensiuni	Densitatea vaselor sangvine; numărul vaselor de drenaj; fluxul sangvin, hipersonoritate centrală crescută
5.Regresie	Hipopigmentație; piele "ridată"; zona de indurație rămâne subcutanat, palpabilă; se observă relieful venelor din periferie	Pierderea structurii specifice; dispariția a aproape tuturor vaselor centrale; vase de drenaj reziduale în periferie

Fig 1.1. Etapele evoluției Hemangiomului Infantil din punct de vedere clinic și ecografic

Un algoritm de învățare DL poate fi o soluție pentru măsurarea precisă și obiectivă a leziunilor specifice hemangioamelor infantile și predicția evoluției acestora. Astfel medicii pot determina o soluție terapeutică optimă pentru fiecare pacient. Pentru a dezvolta un astfel de algoritm trebuie să stabilim metoda de diagnostic și stadiile de dezvoltare a afecțiunii. Diagnosticul HI începe cu istoricul personal al fiecărui pacient. Confirmarea poate fi făcută prin ecografie Doppler sau RMN. Pentru un diagnostic mai sigur se poate efectua și o biopsie a formațiunii tumorale și colorare la imunohistochimie [9]. Evoluția HI prezintă cinci etape. Acestea sunt identificate prin caracteristicile clinice și cu un ecograf Doppler care codează culorile (Color-Coded Doppler Sonography, CCDS).

Un aspect important al acestei afecțiuni îl prezintă localizarea acestuia deoarece tratamentul poate avea efecte secundare locale sau sistemice cu potențialul de a produce cicatrici desfigurante. Tratamentul intervențional este rezervat numai pentru HI care prezintă pericolul unor complicații funcționale sau estetice. HI care fac parte din această categorie sunt cele din zona facială (periorbital, perioral, zona urechilor, buzelor sau nasului), zonei anogenitale (vulvă, uretră, anus), cele care au o proliferare accelerată, difuză și infiltrativă, indiferent de regiunea



afectată [1]. Acestea necesită luarea unei decizii rapide în privința metodei de tratament însă, HI au tendința de a involua spontan, de aceea supravegherea acestora pe un timp mai îndelungat rămâne o metodă viabilă de tratament în cazul unor HI mici și necomplicate din regiuni care nu prezintă un risc major.

Investigarea unui HI în practica clinică uzuală se face cu ajutorul unei rigle și fotografierea acestuia la diferite intervale de timp. Prin inspecție vizuală se estimează evoluția, sau involuția HI. Se notează procentul din aria HI care devine mai estompat în timp. Acest tip de investigație este subiectiv și pot exista erori de măsurare. Din acest motiv este necesară o metodă de detecție și predicție. Din punct de vedere clinic nu există caracteristici bine cunoscute ce pot afecta viteza de involuție a HI sau prezicerea încheierii acestei etape.

1.3. PROBELE IMAGISTICE FOLOSITE ÎN PRACTICA MEDICALĂ ACTUALĂ

Cea mai comună metodă de diagnostic al unui HI este examinarea clinică [10]. Investigarea imagistică are ca scop confirmarea diagnosticului, caracterizarea leziunilor și determinarea fazei în care se găsește leziunea (proliferare sau regresie) [11]. Cele mai folosite tehnici de imagistică sunt ecografia Doppler și rezonanța magnetică (RMN), în conformitate cu [11] și clasificarea ISSVA [9]. În comparație cu Doppler, investigația RMN oferă mai multe informații despre HI cum ar fi estimarea întinderii leziunii în profunzime, stabilirea raporturilor cu alte elemente nervoase și vasculare importate și pentru a evalua răspunsul la tratament sau succesul unei rezecții chirurgicale [11]. Dezavantajul aceste tehnici îl prezintă necesitatea injectiei unei substanțe de contrast și anestezierii copiilor. În plus, costul reprezintă un obstacol important. Ecografia simplă cu niveluri de gri și Doppler color reprezintă o alternativă mai ieftină. Aceasta s-a dovedit utilă pentru stabilirea dimensiunii leziunii, accentuarea porțiunilor subcutanate care pot fi omise la examinarea clinică și evidențierea hemangioamelor față de alte malformații vasculare. Ecografia este o tehnică neinvazivă, ușor de suportat de copii și mult mai ieftină în comparație cu RMN [12] [13]. O altă soluție o reprezintă termografia prin infraroșu [14]. Această tehnică folosește o cameră cu infraroșu care măsoară temperatura de la nivelul pielii. Cum zonele mai vascularizate vor avea o temperatură mai mare decât regiunea din jur face termografia o tehnică de imagistică utilă în caracterizarea HI. Termografia nu este menționată în clasificarea ISSVA [9], aceasta se află printre mijloacele de investigare imagistică a anomaliilor vasculare, menționată în studii încă din 1976 [15].



Recent un studiu a urmărit evoluția HI cutanate la 102 copii cu vârste între o lună și un an, documentat folosind termografia în infraroșu. În articol s-a constatat că există o diferență de temperatură între regiunea înconjurătoare și HI de 3°C iar în faza de involuție temperatura HI scade treptat. S-a considerat regresia completă când diferența de temperatură dintre HI și piele a scăzut sub 0.5°C . În acest studiu s-a folosit termografia pentru a monitoriza evoluția hemangioamelor (proliferare și regresie) și a decide repetarea sau nu a ședințelor de tratament. Termografia, fiind o tehnică non-contact s-a arătat utilă pentru investigarea HI în zone mai sensibile (pleoapă, nas, zona genitală). În studiul [16] s-a urmărit evoluția HI în tratamentul cu propranolol. Autorii au înregistrat progresul diferenței de temperatură între zona HI și piele la două grupuri de 6 respectiv 28 de copii. S-au observat scăderi ale mediei de temperatură de la 1.5°C înainte de tratament. Pe baza vitezei de scădere a temperaturii s-a putut aprecia durata următoare a tratamentului.

1.3.1. Stadiul actual al sistemelor pentru diagnoza asistată de calculator (cad) a leziunilor

Creșterea puterii de calcul a calculatoarelor din ultimii ani a permis dezvoltarea algoritmilor de inteligență artificială (AI) cu o puternică influență în domeniul procesării imaginilor medicale [17]. Tot mai multe domenii ale practicii medicale includ diagnoza asistată de calculator (en: Computer Assisted Diagnosis, CAD) [18]. Răspunsul oferit de un astfel de algoritm este o a doua opinie menită să ajute cadrul medical în identificarea anomaliilor și a cuantificării progreselor unor boli. În cazul unor leziuni mai complexe unde eroarea umană este un factor, AI va ușura etapele de detecție și caracterizarea a leziunii prin completarea capacităților medicului și reducerea timpului necesar unui diagnostic precis [17].

Un sistem CAD este format din un modul pentru achiziția și pre-procesarea imaginii, un alt modul pentru detecția regiunii de interes (en: **Region Of Interest ROI**) apoi se extrag și se selectează trăsăturile ROI și nu în ultimul rând clasificarea și evaluarea ROI. Pentru un astfel de sistem cea mai importantă etapă este cea de preprocesare a datelor. În general imaginile medicale sunt foarte diferite între ele. Acestea trebuie aduse la un format standard cu care algoritmul a fost antrenat [19]. În plus preprocesarea înseamnă și îmbunătățirea calității imaginilor prin reducerea zgomotului și a artefactelor, îmbunătățirea muchiilor și a contrastului etc.. Eficiența identificării ROI și extragerea de trăsături depinde foarte mult de calitatea imaginilor utilizate [17].



Există două modalități de detectare a ROI. Acestea sunt împărțite în două categorii: manuale și automate. Pentru cele manuale este necesară interacțiunea medicului, în principal pentru segmentarea imaginii. Cele automate pot face pre-procesarea imaginilor (cum ar fi segmentarea) fără asistența unui medic. Extragerea de trăsături înseamnă extragerea diferitelor elemente caracteristice ale ROI care sunt folosite de obicei pentru a lua o decizie referitoare la patologia unei structuri sau țesut (de exemplu, culoare, textură, simetrie, margini, regularitate etc.). După extragerea de trăsături algoritmul va putea clasifica ROI în funcție de antrenamentul pe care l-a primit. Cu cât trăsăturile sunt mai calitative cu atât se îmbunătățește precizia de clasificare. Metodele de selecție a trăsăturilor utilizează strategii de căutare, care sunt împărțite în trei categorii: exhaustive, euristice sau non-deterministe [17]. Pentru a antrena un algoritm este necesară o bază de date cu patologii cunoscute, indicate de către specialiști. Apoi algoritmul are sarcina de a clasifica imaginile pe baza trăsăturilor extrase. Acuratețea acestuia este verificată de fiecare dată fiind comparată cu răspunsul dat de specialiști. Algoritmul își reglează singur parametrii cu scopul de a minimiza eroarea.

Sistemele dezvoltate în prezent [20] [21] [22] pentru diagnoza afecțiunilor pielii prezintă rezultate promițătoare pentru diagnoza asistată de calculator a cancerului de piele pe baza imaginilor termografice, dermoscopice, respectiv ecografice. Un studiu comparativ al mai multor clasificatoare arată că acestea sunt capabile să stabilească dacă o formațiune tumorală este benignă sau malignă, prin imagini RMN [15]. Deși CAD este deja o parte a evaluării clinice a unor afecțiuni, aplicațiile sale în diferite tipuri de leziuni ale pielii se află încă într-un stadiu incipient.

1.3.2. Stadiul actual al sistemelor pentru evaluarea asistată de calculator a hemangioamelor infantile

În prezent evaluarea stadiului de evoluție a unui HI este făcută vizual. De aceea este nevoie de un sistem de evaluare și monitorizată de calculator. Acesta este un obiectiv ce trebuie urmărit cu atenție. În momentul de față sunt raportate foarte puține rezultate referitoare la acest subiect [23] [16]. În [23] este prezentată singura metodă raportată până în prezent pentru monitorizarea automată a regresiei HI folosind un cameră foto digitală uzuală pentru preluarea datelor. În articol, algoritmul are rezultate rezonabile dar nu suficiente ca precizie. Segmentarea poate avea erori semnificative.



În alt articol, [16] se studiază regresia HI prin imagini termografice dar autorii nu oferă detalii despre realizarea și caracteristicile tehnice ale software-ului folosit pentru monitorizare.

1.4. REȚELE NEURONALE CONVOLUȚIONALE, DEEP NEURAL NETWORK

Pentru procesarea de imagini cei mai folosiți algoritmi sunt rețele neuronale convoluționale (CNN). Sunt o unealtă complexă și adaptivă care poate clasifica un număr mare de imagini eficient. Până nu de mult un astfel de algoritm avea nevoie de o putere de calcul mult prea mare pentru a fi eficient (având în vedere faptul că presupune multe calcule aritmetice, cum va fi explicat în continuare). Însă datorită dezvoltării hardware, a unităților de procesare grafică (en: **Graphics Processing Unit**, GPU) CNN-urile dau randament și sunt folosite în foarte multe domenii.

Rețelele neuronale își iau numele de la asemănarea funcționării lor cu cea a sistemului nervos animal. În principiu un neuron preia impulsuri electrice de la neuronii din jur și, pe baza acestora, decide dacă transmite mai departe impulsul electric și către cine. Într-un CNN datele de intrare sunt procesate de fiecare strat de neuroni (numit și en: „layer”) și mai departe este transmis un semnal pentru a fi procesat de următorul layer. Ultimul strat are de obicei sarcina de a clasifica datele de intrare, bazat pe calculele efectuate de straturile anterioare.

Diferența dintre un algoritm tradițional **Machine Learning** (ML) și o rețea neurală adâncă (sau „Deep Neural Network”, DNN) este numărul de straturi. ML are în jur de două trei layer-uri în schimb un DNN poate avea sute. Ce face DNN state-of-the-Art este acuratețea. Acest tip de algoritm a reușit să facă anumite sarcini mai bine ca oameni. De exemplu o companie Google, DeepMind, a creat un DNN care a reușit să îl învingă pe campionul Lee Sedol la jocul chinezesc Go [24]. Principiul de funcționare este următorul: să spunem că avem un set de imagini care conține patru tipuri de obiecte. Dorim ca DNN-ul să identifice automat ce obiect se află în imagini. Fiecare imagine trebuie să aibă o etichetă (en: „Label”) care îi va permite algoritmului să învețe. Folosind datele de învățare DNN-ul va începe să înțeleagă caracteristicile (en: „Features”) fiecărui obiect și le va asocia etichetei obiectului respectiv. Fiecare strat preia informație de la stratul precedent și îl predă mai departe. Rezultatul sunt filtre care conțin features corespunzătoare obiectelor. Cu cât un DNN este mai adânc, aceste filtre vor fi din ce

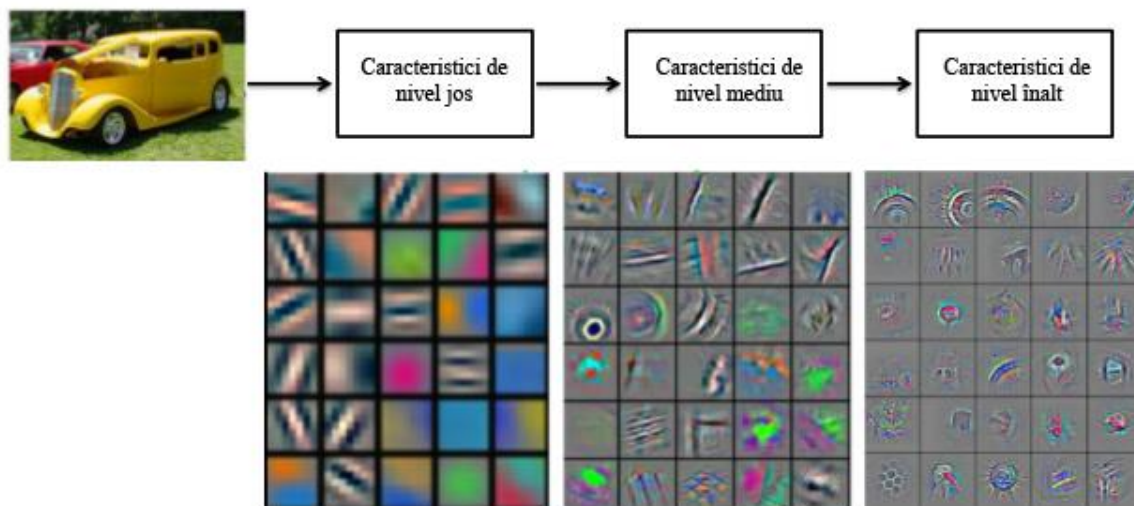


Fig 1.2. Ilustrare a diferitelor straturi de „Features” sau caracteristici din diferite adâncimi ale CNN-ului^[1].

în ce mai abstracte. DNN-ul învață direct din aceste filtre, noi nu avem control asupra ce features sunt învățate. Putem spune că straturile unui DNN fac parte din două categorii. Straturile de detectare a caracteristicilor obiectului și straturile de clasificare, Fig 1.3. și 1.4. „Feature Detection Layers” au trei tipuri de operații: Convoluție, straturi de reducerea a dimensiunii spațiale, și straturi de activare. Straturile de convoluție baleiază un set de filtre pe imagini care activează caracteristici ale obiectului. Pooling, sau stratul de reducere a dimensiunii spațiale are ca sarcină reducerea numărului de parametri care rețeaua trebuie să îi învețe. Rectified Linear Unit (sau ReLU) permite rețelei să se antreneze mai eficient prin aproximarea valorilor negative din filtre la zero. Aceste trei straturi pot fi repetate până la sute de ori, fiecare dintre ele învățând caracteristici ale obiectelor de clasificat.

[1] Vizualizarea de Features a unui CNN antrenat pe setul de date ImageNet de către Zelier & Fergus 2013

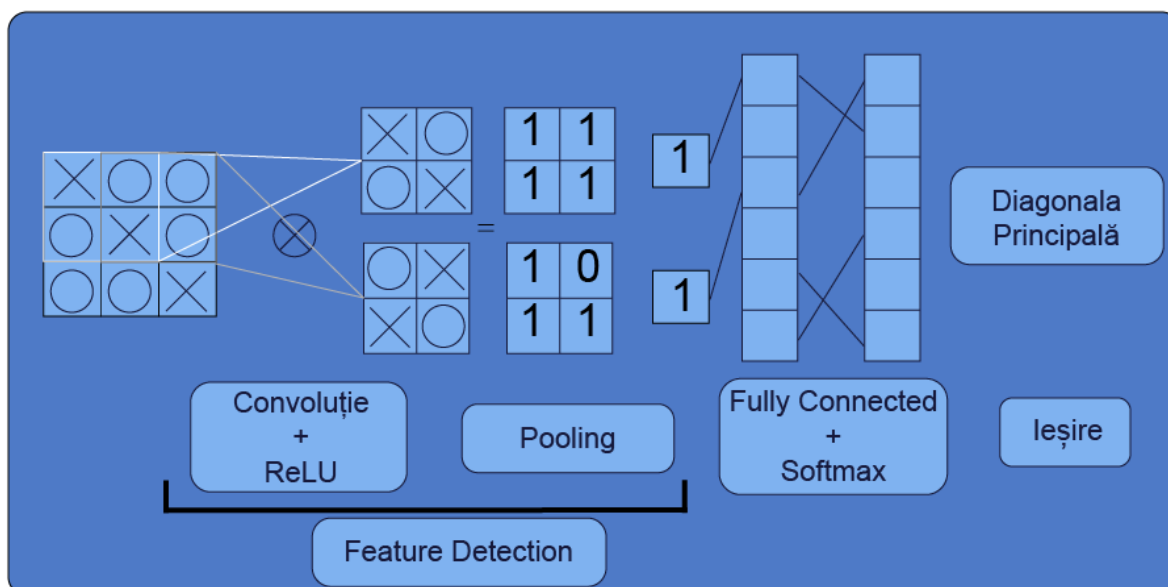


Fig 1.3. Schiță a straturilor dintr-un DNN. Evidențiate sunt straturile care se ocupă cu detecția caracteristicilor imaginilor

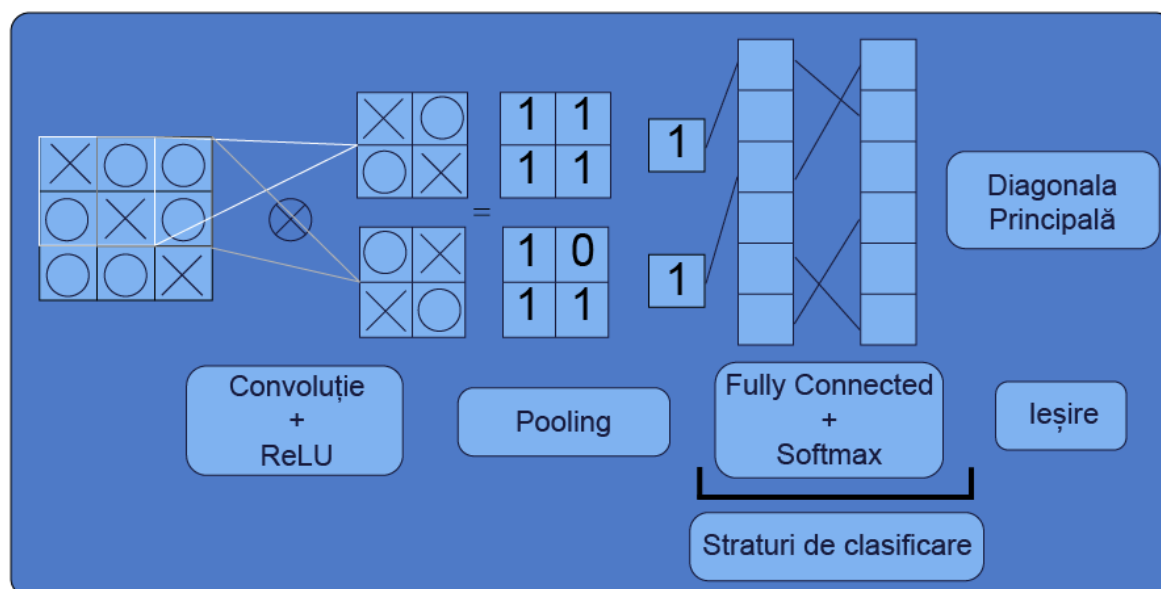


Fig 1.4. Schiță a straturilor dintr-un DNN. Evidențiate sunt straturile care se ocupă cu clasificarea imaginilor

Machine Learning	Deep Learning
Rezultate bune cu un set mic de date	Necesită o bază de date substanțială
Durată a antrenării scurtă	Sarcini computaționale intense
Caracteristicile sunt alese manual	Învăță singur caracteristicile necesare
Acuratețea se atenuează	Acuratețea este infinită

Fig 1.5, Comparație între avantajele și dezavantajele celor două arhitecturi CNN[25]

Straturile care se ocupă cu clasificarea sunt: straturi complet conectate și cele care implementează funcția de cost. Fully connected layer, stratul complet conectat, are ca ieșire un vector de dimensiune K , unde K este numărul de clase care DNN-ul le poate identifica. Acesta conține ponderile de apartenență a unui obiect la clasele respective. Ultimul strat îl reprezintă implantarea funcției de cost care ne va oferi răspunsul DNN-ului.

Deep Learning este un subtip al Machine Learning. Cu ML trebuie extrase caracteristicile imaginilor manual. Cu o rețea neurală adâncă trebuie doar să introduci imaginile iar aceasta va învăța caracteristicile automat. DL are nevoie de o bază de date foarte mare, de la sute de mii de imagini până la milioane pentru cele mai bune rezultate. În plus, presupune un efort computațional intens și necesită un GPU performant.

1.4.1. Starea actuală a cadrelor și librăriilor pentru „Machine Learning”

Clasificarea imaginilor poate fi definită ca sarcina de a împărți imagini în una sau mai multe clase predefinite [26]. Astfel se stabilesc bazele pentru alte domenii ale Computer Vision cum ar fi localizarea, detecția sau segmentarea obiectelor. Deși aceste sarcini sunt foarte ușor de executat pentru oameni, computerele au un mare dezavantaj. Multe din cele enumerate mai sus au o componentă subiectivă, care este greu de descris matematic. De aceea s-a început prin a extrage caracteristici ale imaginilor manual care au fost introduse într-un clasificator automat. Un mare dezavantaj al acestei tehnici a fost că, acuratețea rețelei era dependentă de calitatea caracteristicilor cu care a fost învățată. În ultimi ani modele neliniare care conțin mai multe



straturi au fost demonstrate că pot depăși aceste provocări. Printre acestea CNN-urile sunt cele mai populare. Cu cât rețelele neuronale evoluează în complexitate cu atât cadrele (en: ML Frameworks) și platformele folosite pentru dezvoltarea și implementarea arhitecturilor au evoluat. În continuare vor fi prezentate platformele actuale precum și ToolBox-ul MatLab folosit în această lucrare.

Theano este o bibliotecă pentru Python care îi permite utilizatorului să definească expresii matematice și să le compileze foarte eficient folosind CPU sau GPU [27]. În plus poate calcula automat diferențe simbolice a unor expresii complexe, ignoră variabile care nu sunt necesare în rezultatul final, refolosește rezultate parțiale pentru a evita calcule redundante, simplificări matematice și are o eroare cauzată de aproximările hardware minimă. Theano folosește ca interfață Python. Limbajul de programare este flexibil, puternic și permite utilizatorilor un mod rapid și ușor de interacționat cu datele. Motorul de calcul din Python este destul de slab în materie de viteză și folosirea eficientă a memoriei. Aceste limitări sunt depășite de motorul de calcul Theano care este similar cu bibliotecă NumPy [28] [29], foarte populară printre dezvoltatori, care permite folosirea datelor în n dimensiuni și multe alte funcții cum ar fi: indexarea și calcule elementare (exp, log, sin). Din acest motiv utilizatorilor le este foarte ușor să se adapteze la sintaxa Theano dacă au folosit în trecut NumPy, îmbunătățind eficiența bibliotecii în același timp. Theano este gratis și open-source, cu licență BSD. Se bazează pe o comunitate foarte activă de utilizatori și dezvoltatori din toată lumea. Marea majoritate a proiectelor de acest tip folosesc ca platformă, de distribuție și îmbunătățire, GitHub. Datorită comunității este foarte ușor de pus întrebări și discutat despre Theano, ușurând procesul de învățare pentru începători.

Theano este compus din două tipuri de noduri: Variabile, care reprezintă date, tensori și noduri operaționale, care aplică operațiile matematice. În practică variabilele sunt folosite ca intrări pentru rețele și ca valori intermediare. Variabilele pot fi intrări pentru mai multe noduri operaționale dar pot fi ieșirea unui singur nod operațional. Acestea pot avea un singur tip, pre-determinat de la dezvoltarea arhitecturii. Principalele categorii sunt:

- *TensorType* reprezintă un vector de n dimensiuni, valorile asociate sunt obiecte NumPy *ndarray*



- *CudaNdarrayType* vectori de n dimensiuni în memoria GPU, asociat cu obiecte *CudaNdarray*
- *GpuArrayType* sunt obiecte care corespund unui nou GPU back-end

O arhitectură este de obicei construită prin crearea de variabile simbolice, care corespund intrărilor în rețea. Variabilele trebuie pre-definite la început. Prin funcții Python utilizatorul poate interacționa foarte ușor cu acestea. În spate se creează noduri de operații și variabile. Modulul *tensor* are multe funcții provenite de la NumPy pentru a prezenta o interfață familială utilizatorilor.

Theano a promovat idei pentru calcularea eficientă bazată pe gradient a majorității bibliotecilor actuale de Machine Learning cum ar fi combinarea a unui limbaj de programare de nivel înalt cu kernel-uri optimizate, în special cele care folosesc GPU și computații simbolice a operațiilor. Funcționalitatea, ușurința de folosire și performanța Theano sunt în continuare o țintă pentru îmbunătățirea platformei, unde un rol important îl are comunitatea.

Caffe este un cadru bazat pe o bibliotecă C++ cu legături în Python și MatLab pentru antrenarea și implementarea rețelelor neuronale și a altor modele profunde. Caffe are standardul state-of-the-art prin faptul că folosește CUDA GPU pentru calcule, procesând până la 40 de milioane de imagini pe zi pe un NVIDIA Titan GPU. Autorii cadrului mai spun că permite antrenarea prototipurilor de rețele atât pe PC-ul unde este scris cât și pe cloud [30]. Caffe pune la dispoziție un Toolkit pentru antrenarea, testarea și dezvoltarea rețelelor neuronale. Softul este dezvoltat de la început pentru a fi cât mai modular posibil, permițând extensia la noi tipuri de date sau straturi. Multe layer-uri sau funcții de cost sunt deja implementate. Modelele în Caffe sunt scrise ca fișiere de configurare folosind limbajul „Protocol Buffer”. Caffe suportă arhitecturi de forma: scheme aciclice. Înaintea începerii compilării, Caffe rezervă memoria necesară în GPU sau CPU. Schimbarea între acestea se face doar cu apelarea unei funcții. Există și legături cu Python și MatLab pentru a ușura interacțiunea cu cod existent.

Caffe stochează date prin vectori cu patru dimensiuni numiți „blobs”. Blobs permit sincronizarea între CPU și GPU pentru o tranziție între acestea mai ușoară. În practică datele sunt încărcate de pe Hard Disk într-un blob în cod pentru CPU, apelează un kernel CUDA pentru a face calcule pe GPU și transportă blob-ul către următorul layer.



Straturile în Caffé preiau un blob sau mai multe la intrare și au ca rezultat un blob sau mai multe la ieșire. Într-un model Caffé straturile au două sarcini în operarea rețelei: propagarea înainte și înapoi. În propagarea înainte se iau ieșirile straturilor și se dau mai departe. În propagarea înapoi se calculează gradientul în funcție de parametrii intrări, care la rândul lor sunt propagate spre straturile primare. Caffé conține un set de layer-uri, cum ar fi: convoluție, Pooling, ReLU și funcții softmax. Un model Caffé începe cu un strat care încarcă datele de pe Hard Disk și se termină cu un layer care aplică o funcție de cost pentru clasificarea imaginilor. Modelele se antrenează cu algoritmul „stochastic gradient descent”. Datele sunt procesate în loturi care trec prin rețea secvențial. Vitale pentru antrenare sunt: descreșterea ratei de învățare, impulsul sgd, oprirea și continuarea învățării.

În primele șase luni de la lansare, Caffé a fost folosit într-un număr mare de proiecte de cercetare la UC Berkley și alte universități. Facebook [31] și Adobe [32] au colaborat cu Caffé sau cu predecesorul său Decaf și au obținut rezultate state-of-the-art. În plus, autorii au antrenat un model cu toate cele 10 000 de categorii a bazei de date ImageNet [33].

TensorFlow [34] este un cadru în care se dezvoltă și se implementează algoritmi de învățare automată. Programele scrise se pot rula fără probleme pe o mare varietate de dispozitive, cum ar fi tabletele și telefoanele. Sistemul este foarte flexibil și este folosit de mulți cercetători și firme de top. Proiectul Google Brain a început în 2011 să exploreze utilizarea rețelelor neuronale foarte profunde. Cei de la Google au folosit sistemul DistBelief, prima generație a TensorFlow, pentru clasificare nesupervizată [35], reprezentare lingvistică [36] [37] și detecția de obiecte [38] [39]. TensorFlow a fost construit pentru implementarea de rețele neuronale. Calculele în TensorFlow sunt exprimate ca fluxuri de date, și a fost dezvoltat pentru flexibilitatea experimentării cu noi modele în cercetare și să fie suficient de performant pentru antrenare a modelelor eficiente. Pentru rețele neuronale de scară mai mare TensorFlow permite clienților să execute cod prin replicare și executarea în paralel a unui flux de date cu multe dispozitive computaționale care colaborează între ele. Toate acestea se pot face cu un minim de efort.



În TensorFlow fiecare nod are zero sau mai multe intrări care reprezintă o operație. Fluxul de date care parcurg nodurile se numesc tensori, vectori de dimensiuni arbitrare a cărui tip este specificat când rețeaua este construită. Se mai pot construi și „marginii” care nu permit datelor să le parcurgă. Acest lucru le permite utilizatorilor să controleze, de exemplu, memoria maximă folosită.

Concepte de bază folosite în TensorFlow includ: Operațiile și nucleele (en: „kernel”), sesiunile și variabilele. O operație reprezintă un termen abstract, cum ar fi multiplicare de matrice sau adunare. Acestea trebuie specificate la construcția rețelei pentru a permite nodului să realizeze operația. Un „kernel” este o implementare a unei operații care poate fi executată pe un anumit tip de hardware, cum ar fi CPU sau GPU. Biblioteca TensorFlow definește seturile de operații și kernel-urile disponibile, desigur se pot adăuga seturi de operații opționale. Pentru a interacționa cu TensorFlow un client trebuie să creeze ceea ce autorii numesc „sesiune”. Principala operație suportată de interfața sesiunii este „Run”, adică executarea codului scris. Autorii susțin că majoritatea utilizatorilor fac o sesiune, creează o rețea și o execută de milioane de ori prin această operație. Pentru operațiile de machine learning TensorFlow mai are o operație specială numită variabilă. Aceasta „supraviețuiește” în urma executărilor repetate a codului și pot fi asociate nodurilor și actualizate la fiecare trecere prin sistem. Implementarea TensorFlow este formată din client, care folosește o sesiune să comunice un master cu mai multe procese (en: „worker processes”). Avantajul este posibilitatea de a avea componentele pe o singură mașină sau pe mai multe, în funcție de aplicația în cauză. În concluzie TensorFlow este un sistem flexibil născut din experiența reală care stă la baza a peste o sută de proiecte care implică Machine Learning în cadrul companiei Google. Autorii au lansat TensorFlow ca o platformă open-source și speră ca o comunitate să se dezvolte în jurul acesteia, nu numai în cadrul Google.

MatConvNet este un ToolBox MatLab care implementează rețele neuronale convoluționale pentru aplicații în domeniul Computer Vision, și înțelegerea imaginilor. Spre deosebire de celelalte librării existente, prezentate mai sus, MatConvNet este un ecosistem prietenos și eficient care poate fi folosit de cercetători în investigațiile lor. Cu un design simplu MatConvNet conține funcții simple, care implementează operații de convoluție sau ReLU, direct în MatLab. Funcțiile sunt construite ca niște blocuri care pot fi folosite pentru a dezvolta



rețele neuronale foarte ușor. De obicei nici nu sunt necesare cunoștințe de limbaj C. MatConvNet este compus din blocurile computaționale, un set de rutine optimizate pentru construcția CNN-urilor, implementate printr-o linie de cod. Mai conține și exemplele și modele pre-antrenate care pot fi folosite așa cum sunt. Comunitatea și aici are o mare influență asupra librăriei, ajutând la dezvoltarea acesteia. Documentația poate fi găsită pe site sau în manual [40]. Viteza este importantă în orice cadru de dezvoltare a rețelelor neuronale. MatConvNet suportă NVIDIA GPU și include implementări CUDA în toți algoritmii. În plus poate folosi și librăria CuDNN cu câștiguri importante în viteză și eficiență folosirii memoriei. Trecerea la calculul pe GPU se face doar cu funcția MatLab „gpuArrays”. Autorii prezintă mai multe teste de viteză în care AlexNet se antrenează cu 265 de imagini pe secundă, cu 40% mai rapid decât implementarea sa pe GPU inițială și de zece ori mai rapid decât pe CPU și este comparabil cu Caffe.

MatLab este o platformă cu un limbaj de programare bazat pe matrice. Folosit de milioane de ingineri în diferite domenii acesta este împărțit în librării, sau „ToolBox” – uri. Pentru această lucrare s-a folosit Neural Network ToolBox™. Această soluție a fost aleasă pentru a implementa DNN-ul deoarece limbajul MatLab permite dezvoltarea de modele DL prin comenzi simple. Spre deosebire de MatConvNet acest ToolBox este realizat de MathWorks™. În plus acest ToolBox MatLab este de șapte ori mai rapid decât TensorFlow și până la patru ori mai rapid decât Caffe2. Se pot accesa cu ușurință modelele cele mai noi cum ar fi GoogLeNet, VGG-16, Alexnet etc. MatLab suportă ultimele procesoare grafice fabricate de NVIDIA cu suport NVIDIA CUDA® . În Capitolul 2 vor fi prezentate funcțiile folosite în implementarea rețelei neuronale. Versiunea MatLab folosită MatLab 2018b cu licență „Free Trial” iar versiunea procesorului grafic folosit GeForce GTX 1050 Ti, Driver Version: 9.2000, Toolkit Version: 8. ToolBox-ul nu necesită instalare însă dacă nu este pre-instalat poate fi descărcat din MatLab cu ajutorul „add-on explorer”. Pentru CUDA librăria poate fi descărcată de pe site-ul oficial NVIDIA.



CAPITOLUL 2: DEZVOLTAREA REȚELEI NEURONALE

Această secțiune va prezenta noțiuni teoretice cu privire la rețele neuronale convoluționale, precum și implementarea Deep learning în această problemă.

O rețea convoluțională își ia numele de la layer-ul de convoluție care nu poate lipsi. Ceea ce o încadrează în rețele neuronale profunde este faptul că acesta conține mai multe straturi de convoluție cunoscute și ca straturi ascunse(en: hidden layers). Matematic o astfel de rețea se poate scrie:

$$f(x) = f_N(f_{N-1}(\dots f_2(f_1(x; w_1); w_2) \dots; w_{N-1}); w_N) \quad (2.1)$$

Funcțiile $f_i, i = 1, \dots, N$ reprezintă straturile, o astfel de funcție primește ca argument x_i care, în funcție de tipul layer-ului se comportă diferit și are o ieșire x_{i+1} . Fiecare strat are caracteristicile sale, unele straturi au și ponderi care pot fi „învățate”, ajustate în funcție de eroare. Pentru simplificare în formulă coeficienții w_i pot conține caracteristici ale stratului și ponderi adaptabile. În plus un CNN poate avea și alte caracteristici de învățare, cum ar fi rata de învățare (en: „Learning Rate”), acestea sunt denumite hiper-parametrii. Toate acestea influențează procesul de învățarea a unui CNN.

2.1. ARHITECTURA UNUI CNN ÎN MATLAB

Pentru a antrena un CNN cu ajutorul ToolBox-ului MatLab avem nevoie de trei componente. Straturile (layers) care compun CNN-ul, opțiunile care ghidează învățarea și funcția „trainNetwork” care are ca argumente datele de intrare, straturile și opțiunile de învățare. În capitolul ce urmează toate aceste aspecte vor fi prezentate.



Primul Layer dintr-un CNN este cel de preluare a imaginilor. În ToolBox-ul MatLab acesta se apelează cu funcția „imageInputLayer”. Ca argumente ale acestei funcții sunt dimensiunile imaginilor de intrare. Dimensiunile se referă la lungimea și înălțimea unei imagini. De exemplu o imagine color va avea HxWx3 pixeli. În plus funcția va și prelucra imaginile printr-o operație de normalizare (scăderea din imagine media datelor de intrare).

Cum stratul de convoluție este cel mai important trebuie stabilită funcționarea acestuia. Layer-ul înmulțește punct cu punct fiecare pixel din imaginea de intrare cu un filtru care parcurge toată imaginea, apoi salvează rezultatul într-o altă matrice. Se pot folosi un număr mare de filtre de diferite dimensiuni. Ieșirea unui astfel de layer sunt matricele rezultate în urma convoluției. În ToolBox-ul MatLab putem crea un astfel de Layer cu funcția „convolution2dLayer”. Acesta va crea un număr de filtre care vor parcurge imaginea de intrare vertical și orizontal. Va face produsul punct cu punct al filtrelor și intrării, apoi va adăuga un termen „bias”. Un filtru este un set de ponderi. Acesta va fi ales, dacă nu este specificat, după o distribuție gaussiană cu media zero și o deviație standard de 0.01. Valoarea inițială pentru termenul „bias” este zero. Pasul cu care filtrul parcurge imaginea se numește „Stride”. Aceasta se poate specifica în apelarea funcției. Numărul de ponderi corespund dimensiunilor unui filtru HxWxC, unde H este înălțimea filtrului, W este lățimea și C numărul de canale. Numărul de filtre determină câte canale va avea ieșirea stratului de convoluție. Se poate specifica numărul de filtre prin argumentul „numFilters”. Pe măsură ce filtrul parcurge imaginea acesta își formează setul de ponderi și bias formând o hartă de caracteristici (en: „Feature Map”). Numărul de hărți ale unui layer convoluțional este egal cu numărul de canale ale ieșirii stratului. Fiecare hartă în parte are un set de parametrii proprii, deci pentru a determina numărul total de parametrii într-un layer convoluțional folosim formula:

$$(H * W * C + 1) * \text{Numărul de filtre} \quad (2.2)$$

unde 1 este valoarea bias. Atributul „Padding” este folosit pentru a adăuga borduri imaginii de intrare. Acest atribut te ajută să controlezi dimensiunea ieșirii layer-ului. Ieșirea stratului are dimensiunile:

$$\frac{\text{Dimensiunile intrării} - \text{Dimensiunea filtrului} + 2 * \text{Padding}}{\text{Stride} + 1} \quad (2.3)$$



Această valoare trebuie să fie un întreg altfel imaginea de intrare nu va fi parcursă în totalitate. Programul va ignora marginile de jos și dreapta. Numărul total de neuroni într-un feature map este produsul dintre înălțimea și lungimea ieșirii, se mai numește și „Map Size”. Numărul total de neuroni din layer este atunci:

$$\text{Map Size} * \text{Numărul de neuroni} \quad (2.4)$$

De exemplu, dimensiunile imaginilor color sunt 55x55x3. Pentru un layer convoluțional cu opt filtre și o mărime a filtrului de 5x5, numărul de ponderi per filtru este $5 * 5 * 3 = 75$, numărul total de parametrii va fi $(75 + 1) * 8 = 608$. Numărul total de neuroni din layer este $50 * 50 * 8 = 20000$. Se pot modifica și parametrii învățării proprii acestui layer. Dacă aceștia nu sunt specificați algoritmul va folosi opțiunile din structura „trainingOptions”. O rețea neuronală convoluțională poate conține mai multe straturi de convoluție. Acest lucru depinde de cantitatea de date și de complexitatea lor.

Specific ToolBox-ului MatLab funcția „batchNormalizationLayer” se folosește între straturi de convoluție și ReLU pentru a accelera învățarea. Layer-ul prima dată scade media primului lot de imagini din imaginea de intrare și o divide cu deviația standard a lotului. Apoi funcția va deplasa imaginea cu un coeficient β și îl va scala cu un factor γ . Aceștia sunt și ei parametri de învățare care vor fi actualizați în timpul antrenării. Prin normalizarea activărilor și gradientilor care se propagă print-un CNN antrenarea va fi mai ușor de optimizat. Datorită acestei funcții putem crește rata de învățare. În plus normalizarea depinde de imaginile care intră în acel lot, pentru profita de acest avantaj vom folosi și atributul „Shuffle” în parametri de învățare.

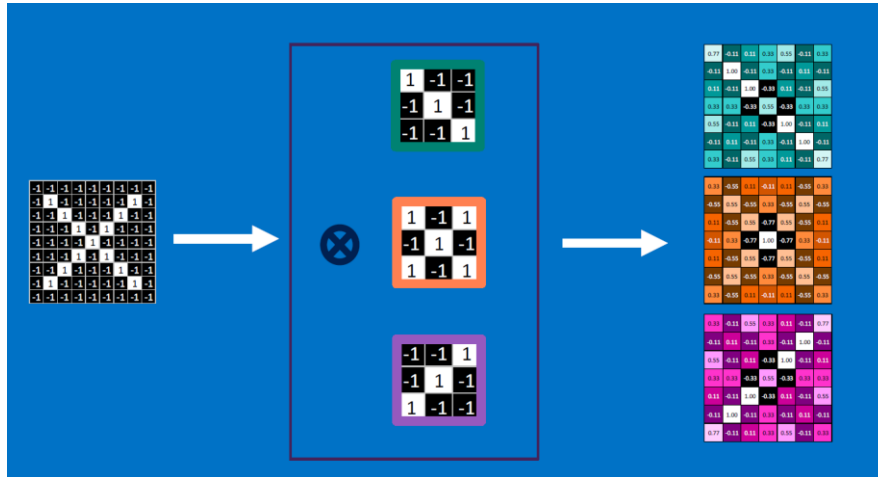


Fig 2.1. Reprezentare schematică a operației de convoluție [41]

Straturile de convoluție și normalizare sunt de obicei urmate de o funcție de activare nelineară cum ar fi „**R**ectified **L**inear **U**nit” sau ReLU. Pentru a crea un astfel de layer trebuie apelată funcția „reluLayer”. ReLU înlocuiește fiecare valoare mai mică decât zero cu zero și nu va schimba dimensiunile datelor de intrare.

$$f(x) = \begin{cases} x, & x \geq 0 \\ 0, & x < 0 \end{cases} \quad (2.5)$$

Există mai multe tipuri de ReLU. De exemplu putem alege ca funcția să permită unor valori negative să treacă, cu o pondere presetată, apelat cu funcția „leakyReluLayer”. Sau dacă dorim să setăm o valoare maximă a valorilor din input atunci apelăm funcția „clippedReluLayer”.

Se mai poate face și o normalizare a răspunsului la nivel local. De obicei urmează după stratul ReLU. Acesta se poate apela prin funcția „crossChannelNormalizationLayer”. Stratul va înlocui fiecare element cu o valoare normalizată în raport cu straturile adiacente, care sunt încadrate într-o fereastră de dimensiune specificată în apelarea funcției. Formula matematică este:

$$x' = \frac{x}{\left(K + \frac{\alpha * ss}{\text{Dimensiunea Fereștei}}\right)^\beta} \quad (2.6)$$

Unde K , α și β sunt hiper-parametrii în normalizare iar ss este suma pătratelor elementelor în fereastra de normalizare [42]. Trebuie specificată fereastra de normalizare și se pot specifica și hiper-parametrii.



Următorul strat este un Layer de Pooling. În ToolBox-ul MatLab acesta se implementează cu funcția „maxPooling2dLayer”. Scopul este de a reduce numărul de conexiuni între straturi. Layer-ul nu învață, însă va reduce numărul de parametri de învățare pentru straturile ulterioare, în plus ajută la reducerea supra-învățării. Un max-pooling layer returnează valoare maximă a regiunii cuprinse în filtru. De exemplu dacă parametri funcției sunt (2,'Stride',2) un filtru de dimensiune 2x2 va parcurge datele de intrare la un pas de 2 pixeli, vertical și orizontal. Dacă mărimea filtrului este mai mică sau egală cu pasul, filtrul nu se va suprapune de la poziție la poziție [43]. Pentru un astfel de layer dimensiunile datelor de ieșire sunt matrice de dimensiuni $n/h \times n/h$ unde n este dimensiunea intrării și h este dimensiunea ferestrei filtrului. Pentru regiuni care se suprapun ieșirea stratului are dimensiunea:

$$\frac{\text{Dimensiunea Intrării} - \text{Dimensiunea Ferestrei} + 2 * \text{Padding}}{\text{Stride} + 1} \quad (2.7)$$

În anumite situații, când rețeaua pare că face prea mult „overfitting”, este nevoie de un layer care va schimba ieșirea unui element în zero cu o anumită probabilitate. Funcția se va apela cu „dropoutLayer”. Pentru fiecare element, „trainNetwork” va selecta un set de neuroni la întâmplare și îi va dezactiva. Acest lucru va construi o arhitectură diferită de epoca precedentă ceea ce va reduce supra-învățarea [42] [44]. Acest layer nu ajută la învățare direct asemănător cu Max Pooling layer.

După toate aceste straturi trebuie să urmeze layer-ul ce va conecta toate ieșirile neuronilor, toate caracteristicile (features) imaginii de intrare și o va clasifica. Funcția se apelează cu „fullyConnectedLayer” iar argumentul său „outputSize” trebuie să fie egal cu numărul de clase în care vor fi clasificate imaginile.

Straturile de ieșire, „softmaxLayer” și „classificationLayer” sunt ultimele straturi din CNN după „fullyConnectedLayer”. Funcția de activare softmax este:

$$y_r(x) = \frac{\exp(a_r(x))}{\sum_{j=1}^k \exp(a_j(x))} \quad (2.8)$$

Unde $0 \leq y_r \leq 1$ și $\sum_{j=1}^k y_j = 1$.

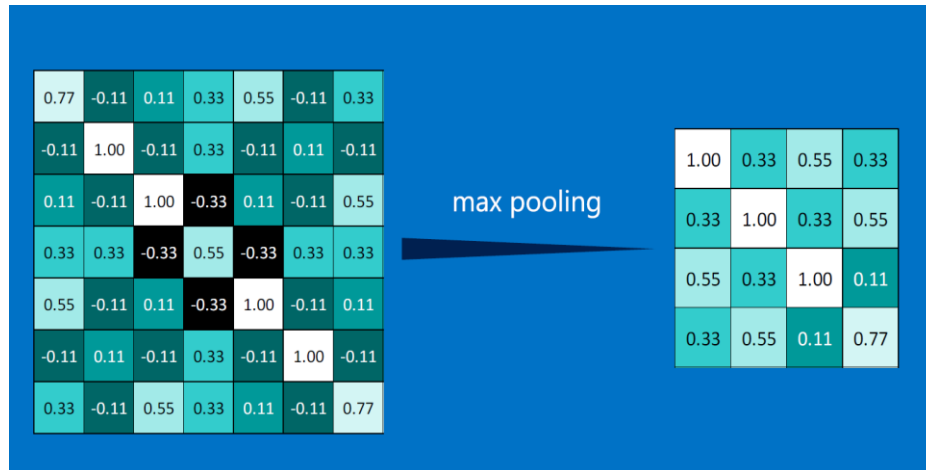


Fig 2.2. Reprezentare schematică a operației de reducere a dimensiunilor spațiale [41]

Funcția softmax mai este cunoscută și ca exponențială normalizată și poate fi considerată ca generalizarea funcției sigmoide [45]. Stratul de clasificare trebuie să urmeze după softmax. Aici „trainNetwork” preia valorile din funcția softmax și le asociază unei clase k .

După ce layer-urile au fost definite următorul pas este specificarea opțiunilor de antrenare pentru rețea. Se pot folosi o varietate de funcții pentru minimizarea erorii. MatLab ne pune la dispoziție „adam” („adaptive moment estimation”), „rmsprop” (root mean square propagation) și „sgdm” (stochastic gradient descent with momentum). În CNN-ul dezvoltat în această lucrare s-a folosit „sgdm”. Aceste funcții actualizează parametrii rețelei prin pași mici către minimizarea erorii sau în direcția opusă gradientului.

$$\theta_{l+1} = \theta_l - \alpha \nabla E(\theta_l) \quad (2.9)$$

Unde l este numărul iterației, α este rata de învățare, θ este vectorul de parametrii iar $E(\theta)$ este funcția de cost. Gradientul funcției de cost $\nabla E(\theta)$ este evaluat folosind tot setul de antrenare. Algoritmul actualizează rețeaua pe baza unui lot de imagini extras din baza de date. Acest lot se mai numește și „mini-batch”. Fiecare trecere printr-un mini-batch se numește iterație iar trecerea prin toată baza de date o epocă. Putem alege dimensiunea lotului și numărul maxim de epoci. Baza de date este amestecată o dată la începutul învățării, însă putem modifica acest aspect cu argumentul „Shuffle”. Algoritmul poate oscila pe o parte abruptă în drum spre



optimizare erorii. Un mod de a preveni această oscilare este de a adăuga un termen de impuls (en: „momentum”, de unde și numele)

$$\theta_{l+1} = \theta_l - \alpha \nabla E(\theta_l) + \gamma(\theta_l - \theta_{l-1}) \quad (2.10)$$

Unde γ reprezintă contribuția gradientului din iterația trecută pentru a introduce un efect de impuls funcției. Putem specifica γ prin atributul „Momentum”.

Un alt aspect important îl reprezintă rata cu care rețeaua învață. Putem specifica acest lucru folosind argumentul „InitialLearnRate”. Funcția de antrenare folosește această valoare pe tot parcursul învățării însă putem opta pentru schimbarea ratei de învățare o dată la un număr de epoci. Putem alege o rata mai mare la începutul antrenării și să o micșorăm pe parcurs. În acest fel putem reduce durata antrenării.

Dacă dorim o antrenare supervizată putem adăuga „ValidationData” în argumentul opțiunilor. Funcția de antrenare va valida răspunsul rețelei o dată la 50 de iterații (default) prin evaluarea imaginilor de validare și calcularea erorii. Antrenarea se oprește automat atunci când rețeaua nu mai învață prin compararea ultimelor iterații. Validarea ne permite să determinăm dacă rețeaua supra-învață (overfitting). Aceasta este o problemă comună, rețeaua în loc să învețe caracteristicile imaginilor aceasta memorează imaginile. Pentru a determina dacă overfitting se întâmplă ne putem uita la graficul acurateței în funcție de epoci. Dacă diferența dintre eroarea de învățare și de validare este foarte mare atunci tragem concluzia că rețeaua face overfitting Fig 4.7. O metodă pentru a reduce acest efect este funcția „augmentedImageSource” care transformă imaginile aleator împiedicând astfel rețeaua să țină minte poziția exactă și orientarea obiectelor.

Componenta Hardware pe care se face antrenarea este selectată automat de către funcția de antrenare. Dacă un GPU compatibil CUDA cu o versiune mai recentă de 3.0 se află instalat în sistem, MatLab îl detectează automat. Dacă dorim putem schimba de pe GPU pe CPU pentru alte tipuri de Machine Learning care nu beneficiază de puterea de calcul al unui GPU.



Ultimul pas înaintea începerii procesului de antrenarea este încărcarea datelor într-o structură compatibilă cu „trainNetwork”. Funcția MatLab „imageDatastore” ne permite stocarea locației imaginilor din hard disk (ex: 'C:\dir\imagedata'). Cu această funcție imaginilor le sunt asociate etichetele de pe directorul în care sunt situate (ex: 'C:\NonHem\NH_1.jpg' va primi eticheta „NonHem”).

În final pentru a antrena o rețea neurală pentru Deep Learning vom folosi funcția „trainNetwork”. Ca argumente sunt baza de date unde se află imaginile, împărțite pe categoriile în care vor fi clasificate, Layer-urile CNN-ului dezvoltat de noi și opțiunile de antrenare. După ce rulăm această funcție, MatLab va începe antrenarea rețelei neuronale.

2.2. PREPROCESAREA DATELOR

Pentru orice tip de Machine Learning preprocesarea datelor este cea mai importantă etapă. Acuratețea și abilitatea de învățare depinde de datele pe care le introducem în CNN. Această etapă este diferită pentru fiecare tip de problemă în parte. Se începe prin alegerea dimensiunilor imaginilor de intrare. Cum setul de imagini este foarte mic pentru antrenarea unui CNN (36 de imagini) trebuie să le transformăm în sub-imagini. Dimensiunea de 55x55 pentru a fost aleasă pentru a permite generarea de cât mai multe sub-imagini. Pentru CNN-ul antrenat în două clase „Hem” și „NonHem” s-au generat 10000 de imagini pentru fiecare clasă. Următoarea problemă urmărită este canalul folosit. Putem intui faptul că folosind un singur canal al imaginilor putem reduce timpul de antrenare. Primul pas în acest sens a fost alegerea canalului a^* din spațiul $L^*a^*b^*$ deoarece acesta conține informații despre culorile roșu-verde, culorile de interes în acest caz. Ceea ce s-a observat este faptul că, folosind acest spațiu de culoare este necesară transformarea imaginilor din RGB în $L^*a^*b^*$, un pas în pre-procesare în plus, iar valorile asociate în acest spațiu nu sunt cele „reale”. Rețele convoluționale sunt făcute cu spațiul RGB la bază, alte valori ar introduce erori în operațiile de convoluție, deși în teorie spațiul a^* ar conține informații mai multe, într-o singură matrice, decât spațiul R. Următorul pas a fost alegerea unui spațiu RGB. Intuitiv spațiul R ar fi trebuit să fie cel mai folosit însă, din modul cum MatLab gestionează imaginile, valorile în sine sunt asemănătoare, în reprezentarea alb/negru, cu pielea. Deci va trebui să alegem din canalele cu mai puțin roșu, pentru a crea contrastul dorit. După cum putem observa în Fig 2.3, canalul G pare a avea cel mai bun contrast.

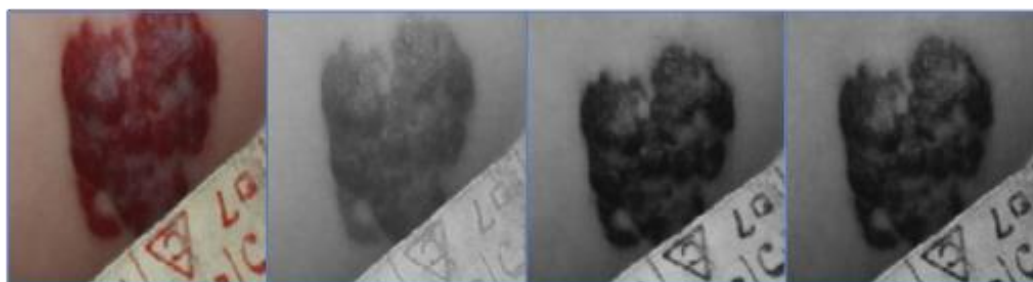


Fig 2.3. Comparație între canalele imaginilor din baza de date înainte de împărțirea în sub-imagini. De la stânga la dreapta: Imagine RGB, canalul R, G și respectiv B.

Pentru a compara câștigul în timp de antrenare, pentru faptul că antrenăm cu un singur canal, au fost făcute experimente pentru ambele tipuri de imagini, color și numai cu canalul G.

Generarea sub-imaginilor a fost făcută în felul următor. Deoarece zona de interes este ușor de încadrat într-un chenar a fost dezvoltată o funcție MatLab prin care utilizatorul va selecta zona de hemangiom. Apoi o altă funcție va decupa imagini de dimensiune 55x55 la un pas dat. În continuare o funcție va verifica fiecare imagine pentru a ne asigura că acestea au dimensiunile propuse. Imaginile au fost salvate apoi în directoare corespunzătoare în format „.jpg”. Funcția „imageInputLayer” din ToolBox-ul MatLab va face o preprocesare a datelor înainte de începerea învățării. Acesta va scădea media imaginilor de intrare din fiecare imagine care intră în CNN. Acest lucru ne scutește de această parte de preprocesare a datelor. În plus layer-ul „batchNormalizationLayer” va face o normalizare similară însă, dacă folosim opțiunea „Shuffle” această normalizare care depinde de imaginile din lotul respectiv, nu va fi la fel pentru fiecare iterație, minimizând supra-învățarea. Algoritmul în pseudocod va fi prezentat în Fig 2.5. Funcțiile vor fi făcute publice pe pagina de GitHub a proiectului după prezentare.

În acest proiect au fost antrenate două CNN-uri. Unul în două clase „Hem” și „NonHem” și unul în trei „S1” „S2” și „S3”. Pentru primul CNN imaginile au fost generate folosind algoritmul prezentat anterior. Hemangiomul a fost încadrat pentru fiecare imagine din baza de date, pentru clasa „Hem” și pielea din jurul HI pentru clasa „NonHem”. Pentru al doilea CNN „S”-urile reprezintă sesiunile de consultație cu medicul în care s-a realizat o fotografie a HI. Numărul reprezintă ordinea în care consultațiile au avut loc. Acestea nu au o durată de timp fixă. Pentru această parte s-a încadrat doar hemangiomul. Pentru această problemă s-au ales numai patru fotografii din setul de 36.



Fig 2.4, Pre-procesarea imaginilor originale prin îmbunătățirea luminozității și contrastului. Stânga este imaginea originală iar în dreapta imaginea după procesarea în Photoshop.

```
Pentru fiecare imagine RGB din setul de antrenare, NonHem,Hem,S1,S2,S3
|   Selecteaza_ROI()    %Se selecteaza zona de interes manual
|   Creaza_subimagini() %Functia decupeaza sub-imagini de dimensiune 55x55
|   Verifica_imaginile() %Verifica daca imaginile au dimensiunea 55x55x3
|   Pentru fiecare sub-imagine
|       Salveaza_canalulG() %Parcurge sub-imaginile si salveaza canalul G al acestora
|       Pentru fiecare imagine salvata
|           Selecteaza_V()    %Selecteaza aleatoriu 20% din imaginile salvate pentru validare
|           Selecteaza_T()    %Idem pentru testare
```

Fig 2.5, Pseudocod al funcțiilor folosite pentru preprocesarea imaginilor în care se salvează canalul G.

Alese deoarece erau singurele care prezentau trei sesiuni consecutive, în stadii ale dezvoltării HI similare. Îmbunătățirea s-a făcut utilizând programul Adobe Photoshop® cu licență Free Trial deoarece, spre deosebire de MatLab, la un număr mic de imagini ne oferă mai mult control asupra contrastului și luminozității imaginilor Fig 2.4. Astfel a fost posibilă aducerea imaginilor de antrenare și de testare la un numitor comun. Rezultatele sunt în Capitolul 4.

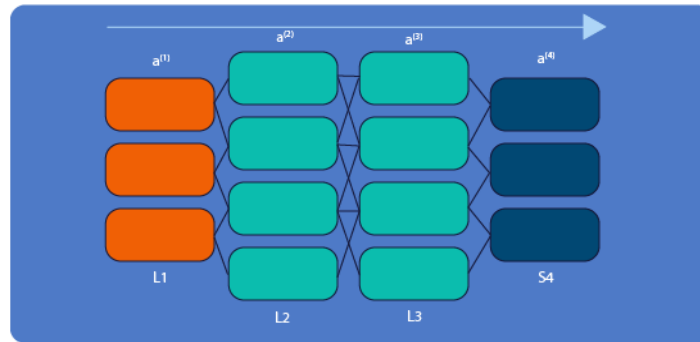
Datele au fost împărțite în seturi de antrenare, validare și testare. Setul de antrenare este cel mai voluminos. Acesta este format din 60% din sub-imaginile obținute. Setul de validare conține 20% din imagini iar cel de testare tot 20%. Împărțirea s-a făcut aleatoriu pentru ambele rețele neuronale.



CAPITOLUL 3: ARHITECTURA CNN PROPUȘĂ ȘI IMPLEMENTAREA ACESTEIA

Această lucrare își propune antrenarea a două rețele neuronale convoluționale pentru clasificarea HI în mai multe clase. Primul CNN va clasifica sub imaginile în imagini de piele, sau „NonHem” și hemangiomul propriu-zis, „Hem”. Diferența dintre piele și hemangiom este destul de accentuată, după cum observăm și în Fig 4.1. Nu va fi complicat pentru un CNN să distingă aceste două clase, deci va avea o arhitectură de bază. O altă rețea va avea sarcina de a clasifica HI după sesiunile de consultații cu medicul. Adică va trebui să determine cât de avansată este afecțiunea. Baza de date conține imagini cu HI preluate la diferite intervale de timp. Acestea urmează dezvoltarea prezentată în Fig 1.1, deci CNN-ul va avea doar date despre HI care involuează în timp. Putem considera clasele S1, S2, S3 ca intervale de timp în care se încadrează HI. De exemplu S1 reprezintă perioada de proliferare a HI iar S2 și S3 perioada de maturare și respectiv regresie. În acest capitol vor fi prezentate arhitecturile celor două rețele precum și opțiunile de învățare.

Antrenarea rețelelor se va face în mod supervizat. Pentru a înțelege diferența dintre o antrenare supervizată și una nesupervizată să presupunem un CNN cu patru straturi L1, L2, L3 și L4. Unde L2 și L3 sunt straturile „ascunse”, L1 este stratul de intrare și L4 stratul de ieșire. Vom nota cu a_n^l , activarea neuronului n din stratul l (de exemplu, pentru stratul L1: $a_1^{(1)}, a_3^{(1)}, a_3^{(1)}$ sunt cele trei intrări). În stratul L2 se evaluează funcția sigmoid pentru produsul activărilor din stratul L1 și se obțin activările din stratul L2. La acești termeni se adaugă și un bias, egal cu 1 ($a_0^{(2)}$).



²Fig 3.1. Propagarea înainte pentru CNN cu patru straturi (L1,L2,L3 și L4)

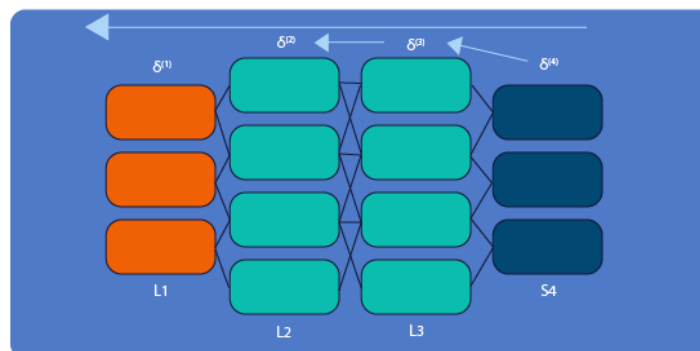
Funcția prin care se obțin activările este:

$$a^{(2)} = g(a^{(1)}\theta^{(1)}), \quad g(x) = \frac{1}{1 + e^{-x}} \quad (3.1)$$

Se procedează în mod similar pentru toate straturile. Această parcurgere al CNN-ului se mai numește „Forward Propagation” sau propagare înainte. Pentru a valida acuratețea fiecărui strat trebuie parcurs CNN-ul și înapoi „Backward Propagation”. Pentru fiecare neuron se calculează eroarea clasificării. Aceasta este diferența dintre rezultatul obținut și adevărata valoare pentru intrare (y): $\delta^{(4)} = a^{(4)} - y$. Straturile vor fi parcurse în ordine inversă, începând cu ultimul strat L4. Pentru straturile L3 și L2 δ se va calcula:

$$\delta^{(3)} = \theta^{(3)}\delta^{(4)}.*g'(a^{(3)}\theta^{(3)}) \quad (3.2)$$

$$\delta^{(2)} = \theta^{(2)}\delta^{(3)}.*g'(a^{(2)}\theta^{(2)}) \quad (3.3)$$



³Fig 3.2. Propagarea înapoi



Dacă acuratețea de antrenare fiecărui lot de imagini este mult mai mare decât acuratețea validării, sau vice-versa, CNN-ul recunoaște foarte bine imaginile cu care a fost antrenat, însă la un set nou de imagini nu are performanțe la fel de bune. S-a ales acest tip de CNN deoarece baza de date este foarte mică și este compusă din sub-imagini. Astfel de baze de date sunt supuse supra-învățării datorită similitudinii datelor de intrare [46] [47].

3.1. CNN ÎN DOUĂ CLASE

Alegerea arhitecturii nu are o rețetă exactă. Numărul de neuroni și de straturi se alege pe baza unor direcții generale. Problema în două clase este una ușor de învățat pentru un CNN deoarece diferența între hemangiom și piele este foarte mare. Pentru acest CNN arhitectura este una de bază. Primul strat este cel de introducere a imaginilor în CNN „imageInputLayer”. Acesta are ca argumente dimensiunea imaginilor de intrare 55x55x1 (sau x3 pentru setul de date color). Imediat după este layer-ul de convoluție. Pentru că această problemă este ușoară un număr de 5000 neuroni a fost ales. Adică 20 de filtre de mărime 5x5. Un strat de convoluție va fi întotdeauna urmat de „batchNormalizationLayer” și „ReLU Layer”. Straturile au scopul de a reduce fenomenul de overfitting. Nu au nevoie de parametri speciali ci doar sunt scrise funcțiile în structura cu layer-uri. În continuare se află un strat de reducere a dimensiunii spațiale „maxPooling2dLayer”. Acesta are ca argumente o fereastră de 2x2 și un pas de 2 pixeli. Stratul complet conectat va avea conexiuni cu toate activările din stratul precedent. Va avea ca argument numărul de clase în care vor fi clasificate imaginile, în acest caz două. Pen-ultimul strat este cel care aplică funcția de cost, fără parametri speciali. Ultimul strat obligatoriu este cel de clasificare care va da răspunsul bazat pe activările tuturor straturilor precedente.

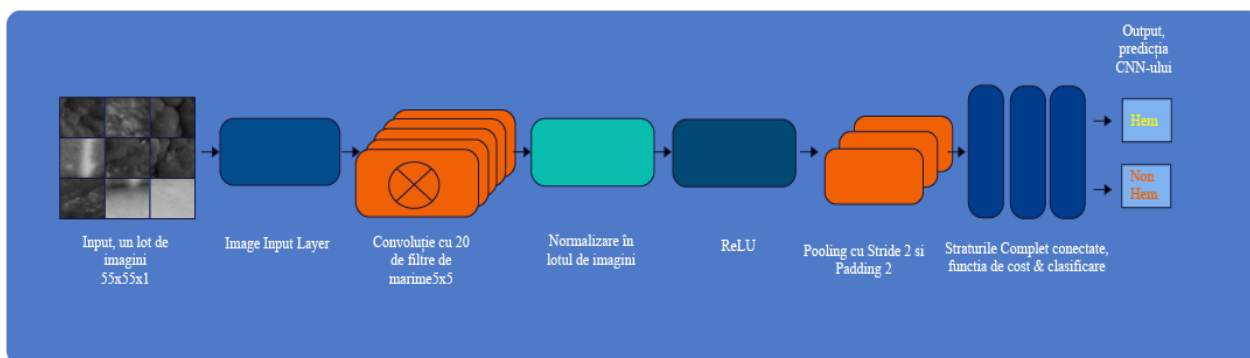


Fig 3.3. Reprezentare schematică a arhitecturii CNN-ului folosit pentru clasificarea în două clase

Arhitectura CNN-ului pentru problema în două clase este (Fig 3.3):

- Un strat convoluțional;
- Un strat de normalizare pe fiecare lot;
- Un strat de activare ReLU;
- Un strat de reducere a dimensiunilor spațiale
- Un strat care apelează funcția „softmax” și un strat complet conectat
- Stratul de clasificare

S-au ales un număr variabil de epoci și s-a observat că acuratețea CNN-ului crește pe măsură ce numărul de epoci crește. Însă acesta tinde să se plafoneze după un număr de epoci iar CNN nu mai învață în plus. Imaginile vor fi amestecate la începutul fiecărei epoci cu perechea de argumente „Shuffle” „every-epoch” și este specificată și structura care conține datele de validare. În plus la fiecare epocă imaginile vor fi alterate. Folosind funcția „imageDataAugmenter” imaginile vor fi translatate cu un interval de $[-10; 10]$ pixeli pe orizontală și verticală și vor fi oglindite pe orizontală.



3.2. CNN ÎN TREI CLASE

Creșterea numărului de clase la prima vedere nu pare să îngreuneze problema prea mult. Însă imaginile cu piele au fost eliminate. Astfel rețeaua va compara numai hemangioame cu diferențe minore între acestea. Pentru această problemă a fost necesară dezvoltarea unei arhitecturii mai complexe și mai profunde.

Pentru a crea o arhitectură mai complexă se recomandă cercetarea CNN-urilor deja existente. Aceste au fost antrenate pe milioane de imagini și pot clasifica imagini în mii de clase. Analizând mai multe rețele neuronale cum ar fi AlexNet, VGG-16 și GoogLeNet s-a realizat o arhitectură asemănătoare. Din cauza limitărilor hardware și numărului relativ mic de imagini CNN-ul va avea un număr mai mic de straturi decât cele menționate mai sus.

Structura generală a arhitecturii se aplică și aici. Primul strat este cel de introducere a imaginilor care au aceleași dimensiuni 55x55x1 (sau x3 pentru color). Ultimele trei straturi: conectat complet cu trei clase ca argument, funcția de cost și layer-ul de clasificare. CNN-ul conține: două straturi de convoluție, primul cu 96 de filtre de dimensiune 11x11 și cu pasul de parcurgere a imaginii de intrare de patru pixeli. După un ReLU se va aplica funcția de normalizare locală între cinci canale vecine ale ieșirii convoluției. Funcția „crossChannelNormalizationLayer” înlocuiește valoarea activării cu una normalizată la vecinii acesteia. Urmează un strat micșorare a dimensiunilor spațiale cu un filtru de 3x3 și un pas de parcurgere a intrării de doi pixeli. Al doilea strat de convoluție are fereastra de dimensiune 5x5 și un număr de 256 de filtre. Pentru a influența dimensiunea ieșirii acestui layer s-a adăugat un Padding de un pixel pe toate muchiile imaginii și un pas de parcurgere de doi pixeli. După încă un ReLU urmează un alt strat de normalizare locală, între cinci vecini. Straturile de convoluție trei și patru au o dimensiune a ferestrei de 3x3 pixeli și un număr de 384 de filtre, fiecare este urmat de un ReLU. După ultima convoluție se va aplica din nou funcția de reducere a dimensiunilor spațiale cu o fereastră de 2x2 pixeli și un pas de parcurgere de doi pixeli. Înainte de ultimele straturi un layer va elimina aleator 50% din activările neuronilor, „dropoutLayer” pentru a evita supra-învățarea.

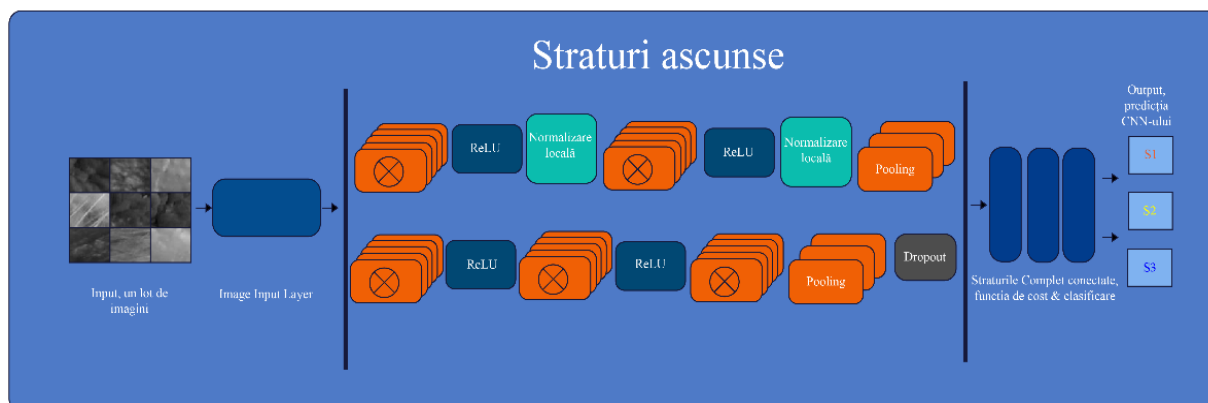


Fig 3.4. Reprezentare schematică a arhitecturii „Deep Neural Network” a CNN-ul folosit în clasificarea HI în trei clase

Arhitectura CNN-ului este (Fig 3.4).:

- Stratul de introducere a imaginilor, de dimensiuni 55x55x1 (sau x3)
- Două straturi convoluționale urmate, fiecare, de un strat ReLU și unul de normalizare locală
- Un strat de reducere a dimensiunilor spațiale
- Trei straturi convoluționale, fiecare urmate de un strat ReLU
- Un strat de reducere a dimensiunilor spațiale
- Un strat de dezactivare a 50% din ieșirile straturilor precedente (dropout)
- Cele trei straturi de clasificare: conectat complet, funcția de cost și stratul de clasificare

Pentru opțiuni a fost setat un număr crescător de epoci pentru a observa îmbunătățirea acurateței arhitecturii. Funcția de antrenare va amesteca datele de intrare înainte de fiecare epocă. La fel ca la problema anterioară datele de intrare au dimensiunea 55x55x1, 1 este canalul G (din RGB) al imaginilor color. Sub-imaginile au fost împărțite în sesiuni. Fiecare sesiune reprezintă o prezentare la medic unde s-a capturat o imagine a hemangiomului. Sesiunile (S1, S2, S3) sunt în ordine cronologică.



CAPITOLUL 4: INTERPRETAREA REZULTATELOR

Cel mai important aspect pe care îl urmărim în antrenarea unei rețele neuronale este acuratețea, câte imagini din setul de test au fost clasificate corect. În domeniul medical acuratețea unor astfel de rețele trebuie să fie relativ mare deoarece răspunsurile au o responsabilitate semnificativă. Cum rețele actuale nu au o acuratețe de 100% întotdeauna va fi nevoie de un medic pentru verificare. În MatLab o funcție va încărca structura „net” care conține ponderile CNN pre-antrenat și îi va calcula acuratețea. În continuare sunt rezultatele celor două rețele neuronale antrenate precum și a algoritmului de clasificare a unei imagini originale.

4.1. REZULTATE

A fost testată acuratețea arhitecturilor în diferite situații. Opțiunile antrenării sunt caracteristicile care influențează procesul de antrenare. Primul lucru de stabilit este rata de învățare. Aceasta se ocupă cu actualizarea ponderilor. O rată de învățare prea mare poate face CNN-ul să sară peste soluție, una prea mică și este posibil să nu găsești niciodată soluția. ToolBox-ul MatLab ne oferă mai multe soluții în acest sens. Putem seta o rată de învățare constantă sau una care se modifică în timp. Avantajul primei metode este viteza de învățare a algoritmului, iar cea de a doua ne permite începerea învățării cu o rată de învățare mai mare, aceasta scăzând în timp. Un alt criteriu de comparare sunt numărul de epoci. Cu cât numărul de epoci este mai mare cu atât acuratețea unui CNN va crește. Această creștere nu este una liniară. Acuratețea învățării ajunge la un plafon la un moment dat, când timpul necesar învățării nu mai justifică câștigul în acuratețe. Alternând acești doi parametri s-a încercat alegerea unui compromis între timpul necesar învățării și acuratețea CNN-urilor.



4.1.1. Clasificarea HI în „Hem” și „NonHem”

În prima parte a experimentelor rata de învățare a fost setată la 0.01 (default MatLab) și numărul de epoci variat. În documentație se recomandă un număr minim de 50 de epoci. În figurile următoare sunt rezultatele obținute după acest experiment. Se observă că timpul de învățare crește semnificativ cu numărul de epoci, dar și acuratețea rețelei. Acuratețea de validare trebuie să se suprapună cu acuratețea de învățare ceea ce ne va demonstra că sistemul nu supra sau sub învață. Numărul de imagini este în total 20200 de imagini din care 50% sunt sub imagini care conțin hemangiom și restul sunt cu piele. Din fiecare clasă s-au selectat aleator, cu o funcție scrisă special pentru acest lucru, imagini pentru testare și validare, mai precis 20% dintre acestea, pentru fiecare clasă. Configurația Hardware folosită pentru aceste experimente este formată din CPU: Intel(R) Core(TM) i5-4690 CPU @ 3.50GHz, 4 Core(s), GPU: GeForce GTX 1050Ti, versiune driver: 398.11, memorie RAM: 16GB, configurația software: Microsoft Windows 10 Pro versiunea 17134 și MatLab 2018b cu licență „Free Trial”. Datele sunt salvate pe un Hardisk.

4.1.1.1. Sub-imagini Color

În spațiul de culoare RGB problema este trivială. CNN poate clasifica imaginile fără probleme, având o acuratețe de 100% cu numai 50 de epoci. Pentru această situație nu s-a mai continuat experimentul deoarece sistemul a ajuns la acuratețea maximă. Clasificarea a durat 5 minute și 10 secunde și a avut o acuratețe de validare de 100%. Din figura 4.1. putem observa diferențele majore între clase. Un CNN simplu, datorită optimizărilor făcute pentru problemele mai complexe, se poate antrena pe un set de date relativ mare de imagini cu diferențe majore între clase, pentru o configurație Hardware comercială, rapid și eficient.

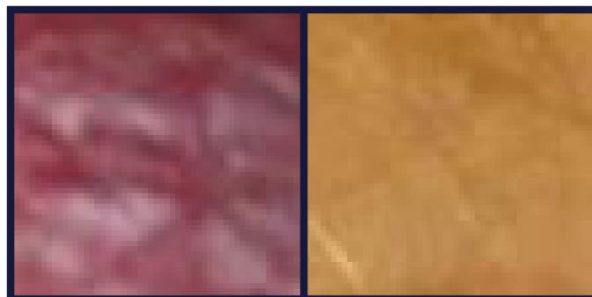


Fig 4.1 Sub-imaginile folosite în color, stânga „Hem” dreapta „NonHem”

4.1.1.2. Sub-imagini în spațiul G

Diferențele între imagini scad atunci când selectăm numai un spațiu de culoare, deci și acuratețea CNN-ului va fi mai mică pentru același număr de epoci. Pentru primul experiment, 50 de epoci, acuratețea rețelei a fost de 88% și antrenarea a durat patru minute și 40 de secunde, cu 30 de secunde mai puțin decât setul de date color. S-a crescut numărul de epoci pentru setul de date G cu următoarele rezultate. La 75 de epoci acuratețea rețelei a fost de 93,50% cu un timp de antrenare de șase minute și 50 de secunde și o acuratețe de validare 99,04%. Chiar și cu mai multe epoci rețeaua nu a reușit să ajungă la performanța anterioară și a depășit timpul de antrenare necesar pentru setul de date color. Mai departe la 100 de epoci rezultatele au fost: o acuratețe a rețelei de 86%, un timp de antrenare de 9 minute și 19 secunde și o acuratețe de validare de 98,38%. Există o diferență destul de mare între acuratețea pe setul de test și acuratețea de validare. Din acest motiv putem spune că sistemul începe să supra-învățe. În acest caz, creșterea numărului de epoci nu va îmbunătăți acuratețea sistemului, trebuie să introducem o rată de învățare dinamică pentru a reduce posibilitatea de supra-învățare a rețelei.

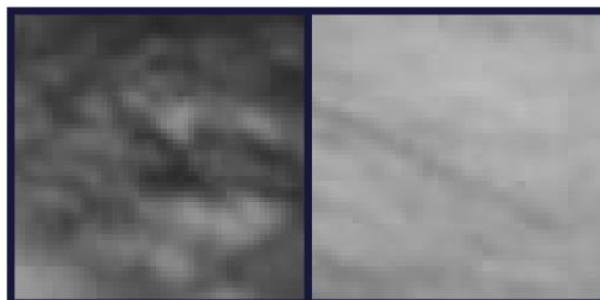


Fig 4.2 Sub-imagini folosite în setul de date cu canalul G, stânga „Hem”, dreapta „NonHem”

Spațiul de culoare,	Numărul de epoci	Acuratețea validării	Acuratețea rețelei	Timpul de antrenare
RGB	50	100%	100%	5 minute si 10 secunde
G	50	97,80%	88%	4 minute si 40 de secunde
G	75	99,04%	93,50%	6 minute si 50 de secunde
G	100	98,38%	86%	9 minute si 19 secunde

Fig 4.3, Ilustrare a rezultatelor obținute în antrenarea unui CNN simplu în problema în două clase. Rata de învățare în acest experiment a fost fixă: 0.01. Aici se poate observa diferența între antrenarea unui singur canal de culoare și antrenarea cu toate cele trei, R, G și B. Arhitectura CNN nu este potrivită pentru clasificarea unui singur canal de culoare, acuratețea cea mai apropiată de spațiul RGB este cea din 75 de epoci însă timpul de antrenare a fost mai mare cu un minut și 30 de secunde.

În continuare s- folosit o rată de învățare dinamică. În structura cu opțiuni se poate specifica rata de învățare inițială și modul în care aceasta se modifică. Selectând argumentul „piecewise” algoritmul va micșora rata de învățare la un anumit număr de epoci. Funcția poate primi ca argument factorul cu care se determină numărul de epoci la care rata de învățare scade, precum și factorul cu care se face scăderea. Pentru acest experiment rata de învățare inițială a fost de 0.01, factorul numărului de epoci 5 și rata de scădere a învățării de 0.2.

Spațiul de culoare,	Numărul de epoci	Acuratețea validării	Acuratețea rețelei	Timpul de antrenare
RGB	50	99,95%	99,93%	5 minute si 18 secunde
G	50	97,78%	85%	4 minute si 43 de secunde
G	75	98,02%	84,50%	6 minute si 50 de secunde
G	100	98,40%	86%	9 minute si 25 de secunde

Fig 4.4. Ilustrare a rezultatelor experimentelor cu rada de învățare dinamică. Aici sunt comparate antrenările cu seturi de date de un singur canal, G, și respectiv trei, R, G și B. în general observăm o mică scădere a acurateței rețelei, și o creștere a duratei de antrenare, pentru toate tipurile de date.

Primul rezultat, la 50 de epoci, a avut o acuratețe de 85% a rețelei, o durată de antrenare de patru minute și 43 de secunde și o acuratețe de validare 97.78%. Rezultatele sunt comparabile cu rata de învățare fixă. Însă, îndată ce începem să creștem numărul de epoci, acuratețea sistemului a început să scadă, la 84,50%.

Din aceste experimente putem spune că, pentru problema clasificării în două clase este mai eficient să folosim tot spațiul RGB chiar dacă în teorie durata de antrenare ar trebui să fie mai mare. Performanțele unei rețele neuronale sunt influențate de tipul de date introduse în rețea și de pre-procesarea aplicată acestora, acesta poate fi un motiv pentru care recomandările din literatură nu se pot aplica în această problemă. Un alt motiv poate fi tipul de normalizare folosit în arhitectura CNN-ului. Timpul mai lung cu 10% decât pe un singur canal este un compromis mic deoarece acuratețea crește cu 15%, ajungând la 100%. Putem deduce din aceste experimente că arhitectura CNN-ului este eficientă pentru setul de date în trei canale R,G și B.



4.1.2. Clasificarea HI după sesiunile de consultare

Pentru clasificarea în clasele: piele și hemangiom, dat fiind diferențele majore între acestea, un CNN simplu a fost de ajuns. Din literatură se poate intuii faptul că asemănarea între imagini și clase necesită o arhitectură mai complexă de rețea neurală. Aici intervin rețelele neuronale profunde. Pentru a justifica creșterea numărului de straturi de convoluție și adăugarea de straturi care combat supra-învățarea aceeași arhitectură simplă de la punctul precedent a fost antrenată cu setul de date în trei clase. Datorită sintaxei simple a ToolBox-ului MatLab a fost necesară doar modificarea argumentului stratului complet conectat pentru ca CNN-ul să poată clasifica trei clase în loc de două. În experimentele următoare rețeaua a fost antrenată cu o rată de învățare de 0.01, fixă. Numărul de epoci a fost crescut treptat, începând cu 50. Setul de date este format dintr-un total de 44,215 de sub-imagini împărțite în trei clase, după sesiunile în care au fost făcute pozele originale. Prima sesiune este formată din 9,000 imagini, a doua din 8,215 imagini și ultima din 9,000. Imaginile de test și validare sunt în număr de 9,000 în total (pentru cele trei clase).

4.1.2.1. Sub-imagini color

Pentru acest set de imagini s-au testat două arhitecturi de rețele neuronale. Cel simplu, utilizate și la experimentele anterioare și unul mai complex, inspirat din arhitecturile state-of-the-art.

Pentru 50 de epoci acuratețea rețelei a fost de 90,56%, acuratețea validării de 90,55% și antrenarea a durat 11 minute și 58 de secunde. Un rezultat acceptabil dar, încercând să creștem numărul de epoci acuratețea nu s-a îmbunătățit fiind de 89,11%. Din graficul epoci în funcție de acuratețe (Fig4.7) se observă că acuratețea de validare are valori mai mici decât cea de testare, ceea ce ne indică faptul că rețeaua supra-învăță. În continuare, numărul de epoci a fost crescut la 100 iar rezultatele au fost: 89,60% acuratețea rețelei, 88,94 acuratețea validării și a durat 25 de minute și 56 de secunde. Se observă că modificarea parametrilor de învățare nu ajută la creșterea acurateței rețelei. De aceea a trebuit dezvoltată o nouă arhitectură mai eficientă.

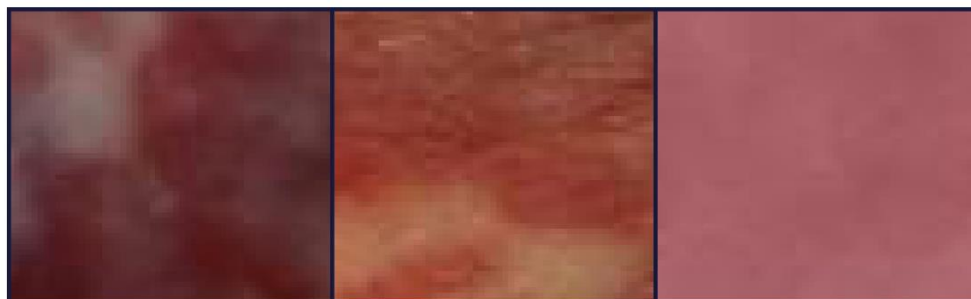


Fig 4.5. Sub-imagini folosite pentru antrenarea în trei clase. De la stânga la dreapta: Sesiunea 1, Sesiunea 2 și Sesiunea 3.

Numărul de epoci	Acuratețea validării	Acuratețea rețelei	Timpul de antrenare
50	90,55%	90,56%	11 minute și 58 de secunde
75	88,91%	89,11%	19 minute și 4 secunde
100	88,94%	89,60%	25 minute și 56 de secunde

Fig 4.6. Ilustrare a rezultatelor obținute de arhitectura simplă cu setul de imagini în spațiul de culoare R,G și B. Se observă că acuratețea nu se îmbunătățește cu creșterea numărului de epoci.

Pentru arhitectura mai complexă experimentul a fost făcut în aceleași condiții. Pentru 50 de epoci acuratețea rețelei a fost de 99,09% la un timp de antrenare de 15 minute și 16 secunde. Din grafic putem observa că CNN-ul nu mai are overfitting deci arhitectura și-a atins scopul. A fost considerată o acuratețe de acest nivel suficientă pentru a concluda experimentul pe setul de imagini color. În continuare s-a realizat același experiment și pe setul de imagini care conțin numai canalul G.

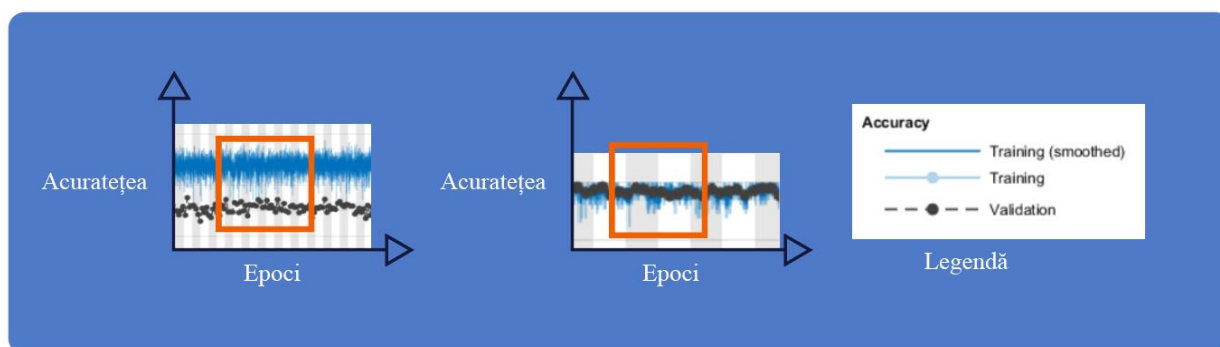


Fig. 4.7. În stânga se află graficul acurateței în funcție de epoci a arhitecturii simple iar în dreapta același grafic rezultat din antrenarea arhitecturii complexe. Chenarul indică diferența dintre acuratețea de validare și cea de antrenare indicând tendința de supra-învățare a arhitecturii. În dreapta este un exemplu în care acestea se suprapun, indicând un comportament corect în timpul învățării.

4.1.2.2. Sub-imagini în spațiul G

În această parte a experimentului CNN-ul folosit a fost cel complex. Imaginile folosite au conținut doar spațiul G . Pentru 50 de epoci: acuratețea rețelei a fost de 81,30% la un timp de antrenare de 13 minute și 52 de secunde. Un timp mai scurt decât setul color de date, așa că numărul de epoci a fost crescut la 75. Acuratețea a crescut puțin, la 83,95% dar și timpul de antrenare a crescut la 18 minute și 28 de secunde. Pentru 100 de epoci acuratețea a ajuns la 88,38% la un timp de învățare de 24 de minute și 56 de secunde. Pe timpul învățării acuratețea de învățare a fost apropiată de cea de validare. Se poate observa o creștere a acurateței sistemului, însă timpul de antrenare crește din ce în ce mai mult. Din aceste experimente se trage concluzia că, deși imaginile color, în teorie, ar dura mai mult să treacă prin CNN, în practică acestea au mai multe informații pe care rețeaua le poate învăța pentru a diferenția clasele.

Numărul de epoci	Acuratețea validării	Acuratețea rețelei	Timpul de antrenare
50	81,81%	81,30%	13 minute și 52 de secunde
75	88,47%	83,95%	18 minute și 28 secunde
100	88,66%	88,31%	24 minute și 56 de secunde

Fig 4.8, Ilustrare a rezultatelor obținute cu arhitectura complexă pe set de imagini care conțin numai canalul G. Se observă o creștere de ~7% în acuratețea rețelei însă avem o creștere a timpului de antrenare aproape dublu.

4.1.3. Clasificarea a unei imagini reale

Scopul acestei lucrări a fost clasificarea HI în cele trei categorii, diferitele stadii ale avansării afecțiunii. Două rețele neuronale au fost dezvoltate în acest sens. Prima având sarcina de a determina ce părți din imagine aparțin regiunii de interes, hemangiomul și background, pielea. A doua are sarcina propriu zisă de a clasifica HI în cele trei momente de timp, sau sesiuni de consultare S1,S2 respectiv S3. Din rezultatele prezentate anterior au fost alese rețelele neuronale cele mai eficiente.

Pentru a realiza acest scop a fost scrisă o funcție în MatLab care are ca argument imaginea de clasificat și ca output va afișa procentul de apartenență a HI din imagine la cele trei clase. Funcția în primul rând va împărți imaginea de intrare în sub imagini de 55 de pixeli. Spre deosebire de pregătirea imaginilor în partea de pre-procesare, filtrele care împart imaginile aici au un pas de 55. Ceea ce înseamnă că sub-imaginile vor avea date unice, și nu se vor suprapune. Sub-imaginile sunt astfel salvate într-un director temporar. Se creează o structură „imageDatastore” care va fi intrarea funcției „trainNetwork”. Rețelele neuronale sunt încărcate și ele în acest timp. Prima rețea va clasifica sub-imaginile și ca crea o structură care conține Label-urile date de funcția „classify”.

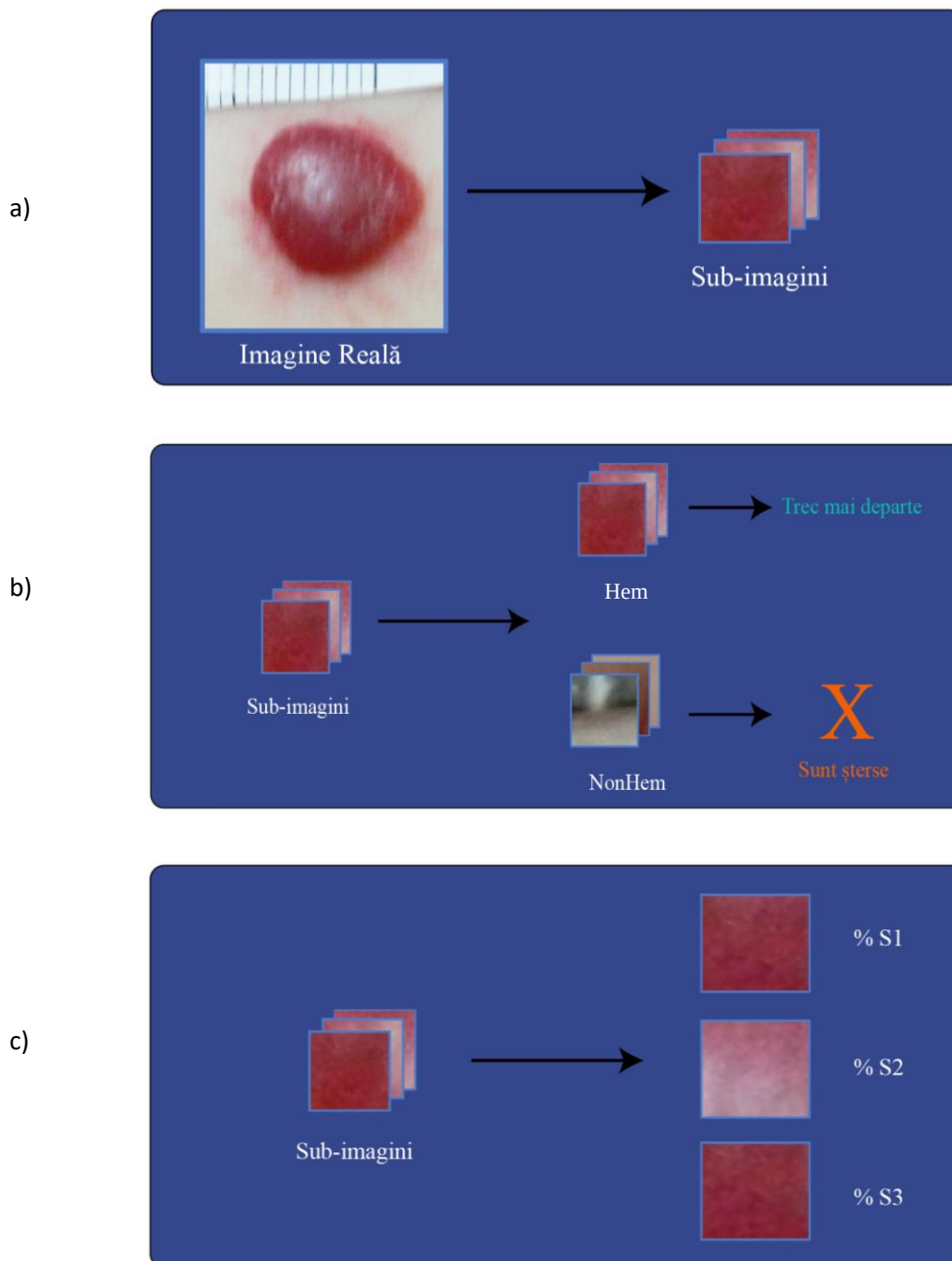


Fig 4.9, a) Imaginea reală este împărțită în sub-imagini. b) Imaginile sunt apoi sortate în piele și hemangiom iar imaginile care nu conțin regiunea de interes sunt șterse din directorul temporar. c) Sub-imaginile sunt apoi clasificate în cele trei clase, urmând ca algoritmul să le afișeze utilizatorului.

Funcția va șterge imaginile clasificate ca „NonHem” deoarece nu aparțin regiunii de interes. Apoi un alt „imageDatastore” este creat pentru a putea clasifica imaginile rămase. Funcția va număra fiecare Label de același fel și se va afișa procentul de apartenență a imaginilor la cele trei clase (unde 100% reprezintă numărul total de sub-imagini care au ajuns în acest pas). Desfășurarea acestui algoritm se află și în ilustrații în Fig 4.9 a) ... c).

Pentru a verifica răspunsul rețelei vom compara imaginile de testare cu imagini folosite la învățare. Din setul de 32 de imagini s-au ales patru care nu au mai multe sesiuni de consultație. Pentru fiecare răspuns al rețelei neuronale s-a comparat regiunea de interes din imagini (Fig 4.10 .. 4.13). Folosind pipeta din Photoshop s-au găsit asemănări între culorile HI din imagini (subliniate cu chenarele din figuri). S-a observat și faptul că unele HI pot fi în mai multe stări de regresie în același timp.

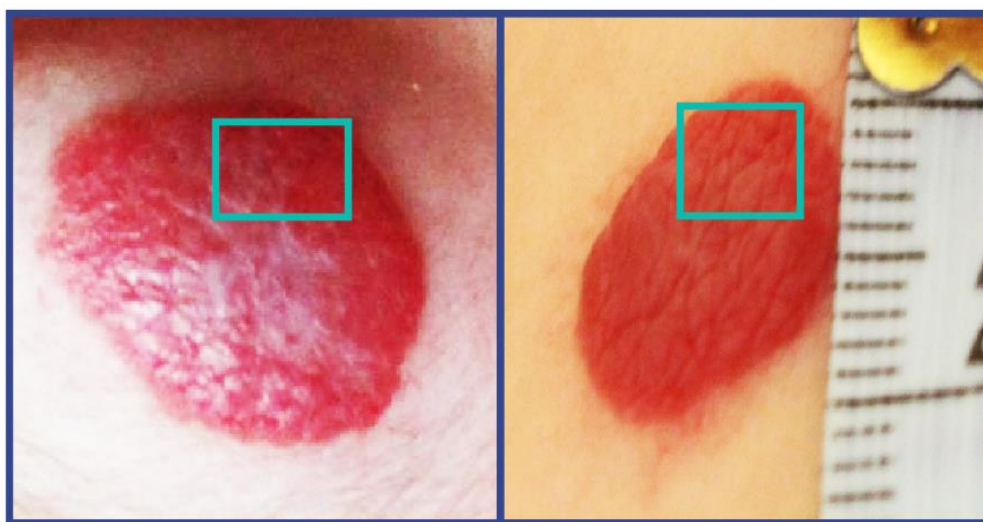


Fig 4.10. În stânga este prima imagine pe care algoritmul a fost testat. Aceasta a fost clasificată ca aparținând tuturor claselor, ceea ce se observă în compoziția HI (chenar). În dreapta este o imagine folosită în antrenare care face parte din sesiunea 3.

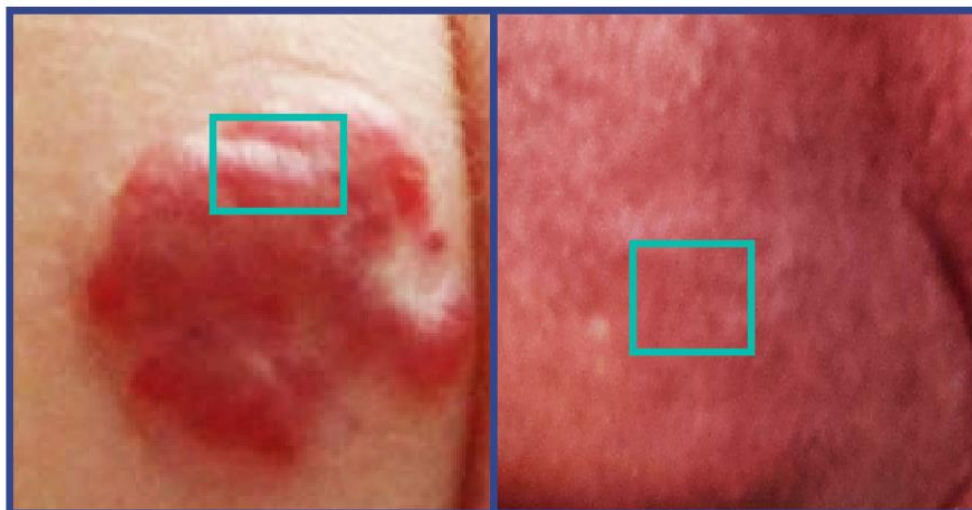


Fig 4.11. În stânga este o imagine clasificată ca aparținând sesiuni unu, se observă asemănările (chenar) cu o imagine de test din categoria respectivă.

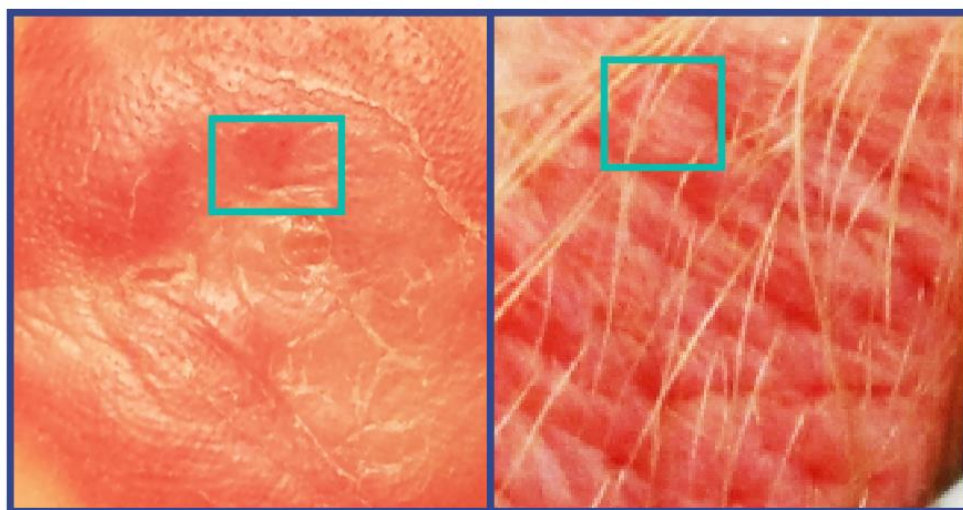


Fig 4.12. A treia imagine folosită pentru testare clasificată ca aparținând sesiunii trei, se observă asemănările (chenar) cu o imagine din setul de antrenare din acea categorie.

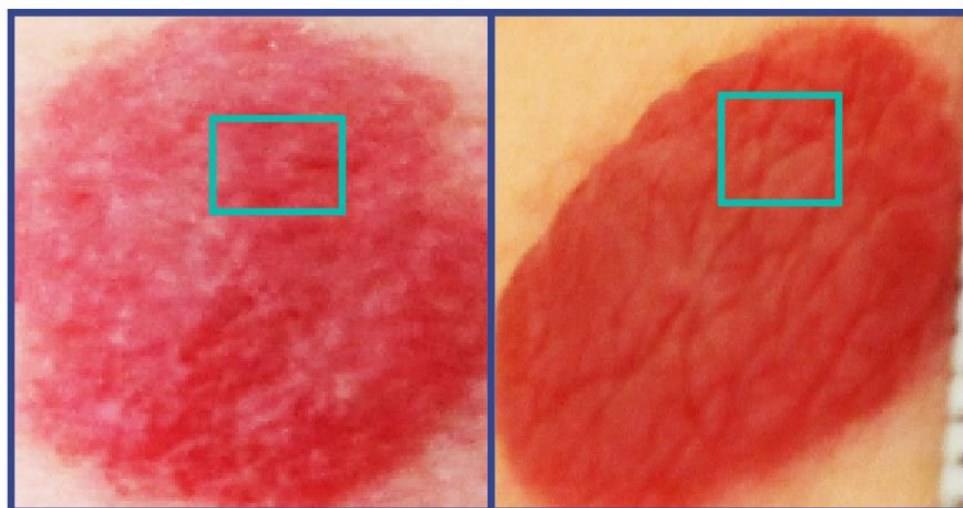


Fig 4.13. Ultima imagine de test folosită a fost clasificată ca aparținând sesiunilor doi și trei. Chenarul subliniază asemănări cu o imagine din sesiunea trei.

Experimentele au fost concepute pentru a evidenția avantajele și dezavantajele folosirii rețelelor neuronale pentru clasificarea HI. S-au documentat diferențele în viteza de antrenare, între folosirea unuia sau a mai multor canale de culoare. S-au modificat opțiunile de învățare pentru a determina numărul optim de epoci precum și rata de învățare optimă, pentru antrenarea în viitor al aceleiași rețele. Overfitting-ul a fost redus la un minim prin introducerea de straturi suplimentare de normalizare și dropout. Și în final un set de imagini care nu au fost folosite în antrenare s-au utilizat pentru a determina aparența la cele trei clase a HI. Pentru a verifica acuratețea clasificărilor s-au comparat imaginile de testat cu imagini care au fost considerate ca aparținând respectivelor sesiuni. În secțiunea următoare sunt prezentate concluziile rezultatelor experimentelor după analiza acestora.

4.2. INTERPRETAREA REZULTATELOR

Resursele Hardware sunt limitate, o rețea neuronală ca să fie eficientă trebuie să aibă rezultate bune la un timp de antrenare cât mai scurt. În această privință s-au implementat multe optimizări ale algoritmilor de învățare, făcând posibilă antrenarea rețelelor de dimensiuni destul de mari încât să fie fiabilă antrenarea lor pe hardware comercial. În dezvoltarea rețelelor prezentate în această lucrare, viteza de antrenare a fost cel mai important aspect. Pentru clasificarea în „Hem” și „NonHem”, în această lucrare, s-a dezvoltat o rețea neuronală simplă, cu un singur strat convoluțional care, din rezultatele obținute, are o acuratețe de 100%. În



realitate aceasta nu va fi niciodată 100%, acest rezultat se datorează diferențelor majore între clase și a faptului că pentru antrenare și validare s-au folosit sub-imagini. Rețeaua a clasificat imagini care se pot compara cu un set de date binare, de zero și unu. Pentru a doua problemă, care a tratat clasificarea HI în trei momente diferite de timp, similitudinea între sub-imagini a fost mai pronunțată. Acest lucru se observă în faptul că folosind o rețea simplă, fără funcții contra supra-învățării, graficele prezintă overfitting (Fig 4.7). Arhitectura rețelei neuronale complexe, prin multiplele straturi de convoluție, normalizare și dropout reușește să atingă o acuratețe de 99% în numai 50 de epoci.

În partea de pre-procesare a imaginilor au fost prezentate diferitele moduri în care un set de date se poate procesa pentru a ajuta o rețea neuronală să învețe eficient. Din experimentele făcute cu diferite opțiuni de învățare și folosind tot spațiul RGB sau numai spațiul G din setul de date, observăm că este mai eficientă folosirea imaginilor color Fig 4.4. Acuratețea unei rețele neuronale ține foarte mult de caracteristicile datelor de intrare. Spre deosebire de alte lucrări [46] care folosesc rețele neuronale pentru detecția de obiecte mici, în imagini cu rezoluție slabă, cum este disertația Mădălinei Savu, unde problema era detecția arterelor din imagini ale fundului ochiului uman, pre-procesarea imaginilor a necesitat îmbunătățiri importante a acestora. În problema din această lucrare nu detecția unui obiect este importantă ci caracteristicile de culoare. În prima instanță s-a considerat un singur spațiu de culoare, cel cu un contrast mai bun, pentru a ajuta rețeaua să învețe mai repede și mai eficient. Din experimente s-a constatat că, deși antrenarea pe același set de date, cu aceleași opțiuni de învățare durează mai mult, câștigul în acuratețe este foarte bun (10% creștere în durată, 15% creștere în acuratețe).

În continuare, pentru a atinge scopul acestei lucrări cele mai bine antrenate rețele neuronale au fost combinate pentru a crea o aplicație practică a acestora. În prima parte imaginea de intrare, reală, trebuie adusă la dimensiunile în care rețelele au fost învățate, adică la 55x55x3. O scalare integrală a imaginii nu este folositoare în acest caz, deoarece rețelele au fost învățate cu sub-imagini. Funcția va separa imaginea originală în sub-imagini. Pielea va fi eliminată și regiunea de interes va fi clasificată în cele trei clase S1, S2 respectiv S3. Așa cum a fost amintit și mai sus, vor exista erori în clasificarea „Hem” și „NonHem” deoarece imaginea reală va conține sub-imagini care conțin hemangiom parțial. CNN-ul îl va clasifica probabil ca



„Hem” însă siguranța lui va fi mai mică de 90%. Pentru a evita selecția imaginilor parțiale un prag al siguranței a fost introdus. Funcția „predict” va indica care sunt procentele apartenenței la cele două clase iar funcția va selecta numai sub-imagini cu o siguranță mai mare de 90%. Acest sistem este pus pentru că CNN-ul antrenat în trei clase a fost antrenat doar cu imagini cu hemangiom, fără piele. Apoi se va aplica funcția „classify” folosind CNN-ul antrenat în trei clase. Nu toate sub-imaginile rămase vor fi clasificate în aceeași clasă, de aceea se va calcula procentul imaginilor care aparțin unei clase, din numărul total de sub-imagini. Deoarece pentru setul de antrenare imaginile alese urmăresc involuarea HI a unui pacient, pe parcursul a trei sesiuni, imaginile care vor fi clasificate pot să nu se cuprindă în aceeași „sesiune” ca imaginile de antrenare. Deci, putem avea imagini care vor aparține altor sesiuni și HI care au o dezvoltare neuniformă. Cu această metodă putem aproxima unde se află afecțiunea ca dezvoltare, indiferent de „sesiunea” sau momentul în timp când pacientul s-a dus la medic și dacă regiunile individuale ale HI se dezvoltă neuniform.

Pentru ultima secțiune a experimentelor interpretarea rezultatelor este mai complicată. Pentru a verifica rezultatele s-au comparat imaginile folosite în antrenare cu cele de test. În Fig 4.10, rezultatul clasificării a fost 33% S1, 20% S2, 46% S3. Aceste procente reprezintă câte din sub-imaginile selectate au fost clasificate în respectivele clase. Asta înseamnă că părți din HI aparțin cel mai mult S3, ceea ce se și observă în centru regiunii de interes, care se mai regăsește și în chenarul din Fig 4.12. Chenarul din Fig 4.10 subliniază o zonă de un roșu puțin mai închis care se regăsește și în imaginile de antrenare. Zona care înconjoară regiunea de interes are o nuanță care se poate regăsi în imaginile din sesiunea unu. Deci un HI poate conține părți în diferite stadii de regresie iar algoritmul a putut identifica acest lucru. În Fig 4.11, este un hemangiom de culoare mai uniformă. În acest caz toate sub-imaginile au fost clasificate ca aparținând sesiunii unu. Subliniat de chenar, se poate observa asemănarea nuanței acestora. În Fig 4.12, este un HI în faza de regresie clasificat corect de către algoritm. Și acesta are o nuanță uniformă pe toată aria, ceea ce a făcut ca toate sub-imaginile să fie clasificate ca aparținând sesiunii trei. În ultima figură din acest experiment, Fig 4.13, este un hemangiom cu colorație relativ uniformă, majoritatea părții superioare pare a fi în faza de regresie, urmată de partea inferioară care se află încă într-un stadiu mediu. Algoritmul a clasificat 85% din sub-imagini ca fiind din sesiunea trei și 15% din sesiunea doi.



În figură chenarul subliniază nuanța care aparține S1 și prin comparație observăm că partea inferioară a regiunii de interes are o nuanță puțin diferită, asemănătoare cu imaginile din clasa S2.

Împărțirea imaginilor originale în sub-imagini poate identifica starea de dezvoltare a hemangiomului în toată aria sa. O parte din HI au diferite nuanțe în ROI, uneori chiar un procent mic (15% în Fig 4.13) unde împărțirea în sub-imagini a putut identifica acest lucru. Un observator uman poate să omită aceste detalii la prima vedere. Acesta este un avantaj important, deoarece algoritmul nu numai grăbește procesul de diagnostic, ci și poate semnala medicului posibile caracteristici ale hemangiomului care pot, sau nu, prezenta un pericol. Un alt avantaj este abilitatea rețelei neuronale de a învăța mult mai multe caracteristici ale HI, cu o bază de date cât mai mare, nu numai identificarea apartenenței la un moment din faza de regresie a afecțiunii. Acestea includ: aspecte ale HI când acesta este pe cale să treacă în fază malignă, sau dacă hemangiomul evoluează în loc să involueze. Se pot introduce astfel orice tip de imagine originală, în orice orientare și cu un procent de piele variabil, deoarece algoritmul va elimina sub-imaginile care nu conțin regiune de interes automat. Acest lucru permite capturarea de imagini de către un utilizator cu orice tip de cameră, de la orice distanță și fără iluminare profesională. Rezultatele se pot salva, iar când pacientul se reîntoarce la consult medicul va avea la îndemână rezultatele sesiunii anterioare cu care poate compara noile rezultate pentru a determina dacă tratamentul își face efectul sau este nevoie de o schimbare. Algoritmul astfel poate semnala anumite tendințe în dezvoltarea HI, de exemplu numai 80% din HI a involuat 20% nu prezintă schimbări majore, rezultate prezentate medicului care poate investiga mai atent porțiunea care nu a răspuns la tratament și poate lua o decizie în continuarea tratamentului.

În faza de selectare a regiunii de interes față de piele și alte obiecte din imagine poate introduce erori în clasificarea ulterioară. Trebuie evaluat dacă o imagine care conține parțial hemangiom merită să fie trecută mai departe sau nu și câtă informație ne permitem să pierdem în acest proces. În acest proiect pragul a fost setat la 90% siguranță totuși, erori s-au strecurat în rezultatele algoritmului. Uităndu-ne la sub-imaginile clasificate ca „Hem” din Fig 4.10, umbra din partea superioară a imaginii conține o nuanță de roșu care a făcut ca un număr de sub-imagini să treacă în faza de clasificare în trei clase, deși acestea nu conțin regiune de interes. După eliminarea manuală a acestor imagini fals pozitive rezultatul nu s-a schimbat cu



mult deoarece numărul acestora a fost mic în comparație cu totalul de sub-imagini. Însă, dacă imaginile au o rezoluție mai mare, în ziua de astăzi cu o cameră comercială ne putem gândii la rezoluții Full HD sau chiar 4K, mult mai multe sub-imagini fals-pozitive se pot strecura prin filtrul impus de primul clasificator, ceea ce va induce erori mult mai mari în clasificarea în trei clase. Acest lucru se întâmplă deoarece a doua rețea, mai complexă, a fost antrenată numai cu regiuni de interes pozitive. Un alt dezavantaj al tehnicii este dependența de rezoluția imaginilor de antrenare. Dacă ne bazăm pe sub-imagini pentru a clasifica un HI avem nevoie de cât mai multe din acesta pentru o clasificare mai precisă. Pentru antrenare s-au produs un număr mare de sub-imagini deoarece un „Stride” mai mic decât dimensiunile sub-imaginilor a fost ales. Deci în setul de antrenare și validare se vor regăsi multe imagini asemănătoare. De aici se trag și rezultatele antrenării de aproape 100% acuratețe pentru majoritatea experimentelor. Un set de date cu rezoluții semnificativ mai mari va putea permite reducerea pasului și generarea de sub-imagini mai puțin suprapuse. În partea de testare nu este recomandată suprapunerea sub-imaginilor deoarece informația va fi redundantă. Un set relativ mare de sub-imagini selectate care conțin regiune de interes va permite algoritmului să descopere detalii și mai fine în faza de clasificare. În experimentele făcute, imaginile originale au rezoluții între 200 și 500 de pixeli, mult mai puțin decât o imagine Full HD care poate oferi și mai multe detalii. Deși scopul acestui algoritm a fost dezvoltarea unui mod de clasificare a HI cu cât mai puține condiții favorabile, se recomandă folosirea unei surse de lumină care să nu se reflecte de pielea pacientului și imagini care conțin cel puțin 80% regiune de interes la rezoluții cât mai mari.

În concluzie această lucrare și-a propus să dezvolte două rețele neuronale care să ajute la clasificarea hemangioamelor infantile în diferitele stadii de evoluție a afecțiunii. Primul CNN a avut scopul de a filtra părțile din imagine care nu aparțin regiunii de interes și de a da mai departe sub-imagini cu hemangiom unui alt CNN antrenat să facă diferența între aceste stadii, numite în această lucrare sesiunii de consultație. Procesul prin care s-a făcut această clasificare a început prin împărțirea imaginii originale în sub-imagini a câte 55x55x3 pixeli fiecare. S-a folosit un pas suficient de mare pentru a nu avea sub-imagini suprapuse. Primul CNN filtrează imaginile care nu conțin ROI iar cel de-al doilea clasifică sub-imaginile rămase. Din rezultate se poate observa că algoritmul va putea identifica procentul din regiunea de interes care aparține fiecărei clase. Acest lucru va oferi medicului informații suplimentare pentru a facilita și accelera în același timp procesul de diagnosticare a afecțiunii.



Universitatea POLITEHNICA din București
FACULTATEA DE INGINERIE MEDICALĂ



Slăbiciunile algoritmului sunt: lipsa unei baze de date mai voluminoasă și rezoluțiile foarte mici, pentru state-of-the-art, ale imaginilor originale. Din fericire, aceste dezavantaje nu prezintă un deficit major și pot fi îmbunătățite în timp. În ultimul capitol al acestei lucrări vor fi prezentate idei pentru dezvoltarea ulterioară în acest domeniu precum și un concept de aplicație de telefon inteligent care va putea fi folosită cu ușurință de personalul medical în condiții reale.



CAPITOLUL 5: DEZVOLTĂRI ULTERIOARE ȘI CONCLUZII

Machine Learning este un fenomen care își lărgeste orizonturile de aplicare pe zi ce trece. Domeniul medical poate beneficia de pe urma rețelelor neuronale. Acestea vor putea accelera consultațiile medicale și alte operații dintr-un spital sau clinică. Vor putea ajuta medicii în diagnostic și alegerea unui tratament pentru multe afecțiuni. Cum imagistica medicală este unul din cele mai importante domenii din medicină rețele neuronale convoluționale vor putea clasifica cu o foarte mare acuratețe multe afecțiuni, ușurând astfel munca medicilor, care se vor concentra pe alte probleme. CNN-urile nu ajută numai în detecție și clasificare dar și în îmbunătățirea imaginilor. Cum definițiile de contrast și calitate a unei imagini sunt diferite de la persoană la persoană, algoritmi pot învăța de la mai mulți medici ce înseamnă o imagine de calitate, pe baza căreia se poate pune un diagnostic. Un dezavantaj al rețelelor neuronale, mai ales cele profunde, îl prezintă necesitatea unei puteri de procesare destul de importante și un set de date foarte mare. Însă, dezvoltatorii de dispozitive mobile au făcut deja progrese în acest domeniu. De exemplu Apple a dezvoltat o platformă incorporată în sistemul de operare iOS, unde se pot dezvolta aplicații care folosesc rețele convoluționale deja antrenate. Pentru identificarea HI de exemplu medicul poate avea o aplicație pe telefon care va procesa imaginea capturată cu camera telefonului și va oferi un răspuns în timp real. Astfel de soluții vor fi fezabile în viitorul apropiat.

5.1. ÎNBUNĂTĂȚIRI VIITOARE

Un utilizator nu ar trebui să cunoască sintaxă MatLab sau să dețină o licență pentru a clasifica imaginile medicale pe care le capturează. De aceea trebuie ca rețeaua neuronală să fie disponibilă publicului larg într-un mod cât mai accesibil. În momentul actual majoritatea

personalului medical deține un telefon de tip smartphone. Transferul rețelelor neuronale din programe ingineresti cum este MatLab, TensorFlow sau Caffé este din ce în ce mai ușor. Cu lansarea ML2 de către Apple în iunie 2018 implementarea CNN-urilor într-o aplicație de telefon nu a fost nicicând mai simplă.

Aplicația poate avea și mai multe funcții. De exemplu poate conține vârsta pacientului, numele sexul, etc. În plus aplicația va putea accesa date despre acest pacient de la sesiunile de consultație anterioare, cum ar fi tratamentul folosit. În plus CNN-ul poate observa dacă tratamentul a fost favorabil sau nu. Medicul poate introduce observații pentru viitor care vor fi reamintite la următoarea sesiune. Toate acestea în timp real, printr-o simplă atingere.



Desigur securitatea datelor pacienților poate fi o problemă, însă acestea nu trebuie să fie salvate pe dispozitiv, ci pot fi salvate în server-ul spitalului printr-o conexiune de internet securizată care va permite numai medicilor să acceseze aceste informații. O astfel de aplicație poate reduce timpul unei consultații semnificativ, reduce erorile umane prin amintirea medicilor a anumitor aspecte ale afecțiunii precum și identificarea modificărilor subtile a colorației hemangiomului.

Fig 5.1. Concept al unei aplicații de telefon mobil pentru analiza hemangioamelor infantile



5.2. CONCLUZII

Implementarea și dezvoltarea rețelelor neuronale în MatLab s-a dovedit a fi un proces relativ simplu. Având cunoștințele necesare dezvoltarea unei rețele complexe inspirată din literatură a fost suficientă pentru clasificarea HI în cele trei sesiuni discutate în această lucrare. Aplicația practică a acestora oferă informații importante despre HI care, deși are caracteristici ușor de observat, aprecierea culorii, ariei și a altor proprietăți, pe o perioadă mai lungă de timp, poate fi dificilă fără o poză precedentă. Pozele care se fac la momentul actual nu sunt standard și nu sunt făcute în mode profesional. Acest lucru poate îngreuna și mai mult distingerea caracteristicilor HI. CNN-ul învățat în această lucrare poate determina, chiar și în situații de lumină slabă sau imagini zgomotoase, stadiul de dezvoltarea a HI fotografiat. Pentru viitor, cu un set de date mai mare, se pot antrena CNN-uri similare pentru a determina dacă un HI este benign sau nu. Se mai poate determina, bazat pe locația HI, dacă acesta va lăsa cicatrici sau urme. Baza de date poate fi și de tip semantic, pentru aprecierea poziției HI pe corp și cunoașterea rezultatelor. Se poate învăța rezultatul diferitelor tratamente și pe ce tip de HI au fost mai eficiente. Pe lângă bazele de date, arhitectura se poate îmbunătăți cu dezvoltării în straturile propriu-zise. Așa cum a fost apariția straturilor „leakyReLU” de exemplu, care nu aplică funcția tuturor activărilor stratului precedent și permite câtorva valori negative să treacă mai departe în CNN, în mod aleator. Domeniul învățării automate a computerelor este în continuă dezvoltare și în fiecare an CNN-urile devin din ce în ce mai eficiente. Acestea ne ajută deja în viața de zi cu zi, în entertainment și în muncă, iar în curând vor constitui o mare parte din domeniul imagisticii medicale.



BIBLIOGRAFIE

- [1] *R. Mattassi. et al*, “Hemangiomas and vascular malformations,” Springer-Verlag Italia, pp. 9–96, 2009.
- [2] *A. Pandey, A. N. Gangopadhyay, S. C. Gopal, V. Kumar, S. P. Sharma, D. K. Gupta, and C. K. Sinha*, “Twenty years’ experience of steroids in infantile hemangiomae” from a developing country’s perspective,” *Journal of pediatric surgery*, **vol. 44**, no. 4, pp. 688–694, 2009.
- [3] *H. Nakayama*, “Clinical and histological studies of the classification and the natural course of the strawberry mark,” *The Journal of dermatology*, **vol. 8**, no. 4, pp. 277–291, 1981.
- [4] *D. W. Metry, C. F. Dowd, A. J. Barkovich, and I. J. Frieden*, “The many faces of phace syndrome,” *The Journal of pediatrics*, **vol. 139**, no. 1, pp. 117–123, 2001.
- [5] *K. G. Chiller, D. Passaro, and I. J. Frieden*, “Hemangiomas of infancy: clinical characteristics, morphologic subtypes, and their relationship to race, ethnicity, and sex,” *Archives of dermatology*, **vol. 138**, no. 12, pp. 1567–1576, 2002.
- [6] *M. C. Finn, J. Glowacki, and J. B. Mulliken*, “Congenital vascular lesions: clinical application of a new classification,” *Journal of pediatric surgery*, **vol. 18**, no. 6, pp. 894–900, 1983.
- [7] *C. Panteliadis, R. Benjamin, H. Cremer, and C. Hagel*, “Neurocutaneous disorders,” *Hemangiomas-a clinical and diagnostic approach*. Anshan Uk, pp. 239–284, 2010.



- [8] C. Léauté-Labrèze, E. D. de la Roque, T. Hubiche, F. Boralevi, J.-B. Thambo, and A. Taëb, “Propranolol for severe hemangiomas of infancy,” *New England Journal of Medicine*, vol. **358**, no. 24, pp. 2649–2651, 2008.
- [9] O. Enjoras, M. Wassef, and R. Chapot, “Introduction: Issva classification,” Cambridge University Press.[Online][Cited: 2009 16-06.] Available at: <http://assets.cambridge.org/9780>, vol. **5218**, p. 48510.
- [10] L. Flors, C. Leiva-Salinas, I. M. Maged, P. T. Norton, A. H. Matsumoto, J. F. Angle, M. Hugo Bonatti, A. W. Park, E. A. Ahmad, U. Bozlar et al., “Mr imaging of soft-tissue vascular malformations: diagnosis, classification, and therapy follow-up,” *Radiographics*, vol. **31**, no. 5, pp. 1321–1340, 2011.
- [11] L. F. Donnelly, D. M. Adams, and G. S. Bisset III, “Vascular malformations and hemangiomas: a practical approach in a multidisciplinary clinic,” *American Journal of Roentgenology*, vol. **174**, no. 3, pp. 597–608, 2000.
- [12] J. e. Dubois, H. Patriquin, L. Garel, J. Powell, D. Filiatrault, M. David, and A. Grignon, “Soft-tissue hemangiomas in infants and children: diagnosis using doppler sonography.” *AJR. American journal of roentgenology*, vol. **171**, no. 1, pp. 247–252, 1998.
- [13] H. J. Paltiel, P. E. Burrows, H. P. Kozakewich, D. Zurakowski, and J. B. Mulliken, “Soft-tissue vascular anomalies: utility of us for diagnosis,” *Radiology*, vol. **214**, no. 3, pp. 747–754, 2000.
- [14] G. Schaefer, M. Závisek, and T. Nakashima, “Thermography based breast cancer analysis using statistical features and fuzzy classification,” *Pattern Recognition*, vol. **42**, no. 6, pp. 1133–1137, 2009.
- [15] F. Desmons, Y. Houdas, C. Deffrenne, and C. Lakiere, “Thermographic study of hemangiomas of children,” *Angiology*, vol. **27**, no. 9, pp. 494–501, 1976.
- [16] B. Kalicki, A. Jung, F. Ring, A. Rustecka, A. Zylak, J. Zuber, P. Murawski, K. Bilska, and W. Wozniak, “Infrared thermography assessment of infantile hemangjoma treatment by propanolol,” *Thermology International*, vol. **22**, no. 3, pp. 102–103, 2012.



- [17] J. Stoitsis, I. Valavanis, S. G. Mougiakakou, S. Golemati, A. Nikita, and K. S. Nikita, “Computer aided diagnosis based on medical image processing and artificial intelligence methods,” *Nuclear Instruments and Methods in Physics Research Section A: Accelerators, Spectrometers, Detectors and Associated Equipment*, **vol. 569**, no. 2, pp. 591–595, 2006.
- [18] K. Doi, “Computer-aided diagnosis in medical imaging: historical review, current status and future potential,” *Computerized medical imaging and graphics*, **vol. 31**, no. 4-5, pp. 198–211, 2007.
- [19] H. Fujita, X. Zhang, S. Kido, T. Hara, X. Zhou, Y. Hatanaka, and R. Xu, “An introduction and survey of computer-aided detection/diagnosis (cad),” in *2010 international conference on future computer, control and communication (FCCC2010)*, 2010, pp. 200–205.
- [20] M. Ogorzaek, L. Nowak, G. Surówka, and A. Alekseenko, “Modern techniques for computer-aided melanoma diagnosis,” in *Melanoma in the Clinic-Diagnosis, Management and Complications of Malignancy*. InTech, 2011.
- [21] S. Kia, S. Setayeshi, M. Shamsaei, and M. Kia, “Computer-aided diagnosis (cad) of the skin disease based on an intelligent classification of sonogram using neural network,” *Neural Computing and Applications*, **vol. 22**, no. 6, pp. 1049–1062, 2013.
- [22] T. M. Buzug, S. Schumann, L. Pfaffmann, U. Reinhold, and J. Ruhlmann, “Skin-tumor classification with functional infrared imaging,” in *Proc. 8th. IASTED International Conference Signal and Image Processing*, ACTA Press, Anaheim, 2006, pp. 313–322.
- [23] S. Zambanini, R. Sablatnig, H. Maier, and G. Langs, “Automatic image-based assessment of lesion development during hemangioma follow-up examinations,” *Artificial intelligence in medicine*, **vol. 50**, no. 2, pp. 83–94, 2010.
- [24] E. Gibney, “Google ai algorithm masters ancient game of go,” *Nature News*, **vol. 529**, no. 7587, p. 445, 2016.
- [25] The MathWorks, Inc., Natick, Massachusetts, United States, “Introducing deep learning with matlab.” [Online]. Available: <http://www.mathworks.com/>



- [26] W. Rawat and Z. Wang, “Deep convolutional neural networks for image classification: A comprehensive review,” *Neural computation*, **vol. 29**, no. 9, pp. 2352–2449, 2017.
- [27] Theano Development Team, “Theano: A Python framework for fast computation of mathematical expressions,” *arXiv e-prints*, vol. abs/1605.02688, May 2016. [Online]. Available: <http://arxiv.org/abs/1605.02688>
- [28] S. v. d. Walt, S. C. Colbert, and G. Varoquaux, “The numpy array: a structure for efficient numerical computation,” *Computing in Science & Engineering*, **vol. 13**, no. 2, pp. 22–30, 2011.
- [29] E. Jones, T. Oliphant, and P. Peterson, “{SciPy}: open source scientific tools for {Python},” 2014.
- [30] Y. Jia, E. Shelhamer, J. Donahue, S. Karayev, J. Long, R. Girshick, S. Guadarrama, and T. Darrell, “Caffe: Convolutional architecture for fast feature embedding,” *arXiv preprint arXiv:1408.5093*, 2014.
- [31] N. Zhang, M. Paluri, M. Ranzato, T. Darrell, and L. Bourdev, “Panda: Pose aligned networks for deep attribute modeling,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2014, pp. 1637–1644.
- [32] S. Karayev, M. Trentacoste, H. Han, A. Agarwala, T. Darrell, A. Hertzmann, and H. Winnemoeller, “Recognizing image style,” *arXiv preprint arXiv:1311.3715*, 2013.
- [33] S. Guadarrama, E. Rodner, K. Saenko, N. Zhang, R. Farrell, J. Donahue, and T. Darrell, “Open-vocabulary object retrieval.” in *Robotics: science and systems*, **vol. 2**, no. 5. Citeseer, 2014, p. 6.
- [34] TensorFlow Development Team, “TensorFlow: Large-scale machine learning on heterogeneous systems,” 2015, software available from [tensorflow.org](https://www.tensorflow.org). [Online]. Available: <https://www.tensorflow.org/>



- [35] A. N. Le, Quoc V, “Building high-level features using large scale unsupervised learning,” in Acoustics, Speech and Signal Processing (ICASSP), 2013 IEEE International Conference on. IEEE, 2013, pp. 8595–8598.
- [36] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *nature*, **vol. 323**, no. 6088, p. 533, 1986.
- [37] O. Vinyals, . Kaiser, T. Koo, S. Petrov, I. Sutskever, and G. Hinton, “Grammar as a foreign language,” in Advances in Neural Information Processing Systems, 2015, pp. 2773–2781.
- [38] A. Frome, G. S. Corrado, J. Shlens, S. Bengio, J. Dean, T. Mikolov et al., “Devise: A deep visual-semantic embedding model,” in Advances in neural information processing systems, 2013, pp. 2121–2129.
- [39] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, A. Rabinovich et al., “Going deeper with convolutions.” *Cvpr*, 2015.
- [40] “MatConvNet, CNNs for MatLab,” [accessed 10-June-2018]. [Online]. Available: <http://www.vlfeat.org/matconvnet/>
- [41] B. Rohrer, “How do Convolutional Neural Networks work?” [accessed 10-June-2018]. [Online]. Available: http://brohrer.github.io/how_convolutional_neural_networks_work.html
- [42] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “Imagenet classification with deep convolutional neural networks,” in Advances in neural information processing systems, 2012, pp. 1097–1105.
- [43] J. Nagi, F. Ducatelle, G. A. Di Caro, D. Ciresan, U. Meier, A. Giusti, F. Nagi, J. Schmidhuber, and L. M. Gambardella, “Max-pooling convolutional neural networks for vision-based hand gesture recognition,” in Signal and Image Processing Applications (ICSIPA), 2011 IEEE International Conference on. IEEE, 2011, pp. 342–347.



- [44] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” *The Journal of Machine Learning Research*, **vol. 15**, no. 1, pp. 1929–1958, 2014.
- [45] N. M. Nasrabadi, “Pattern recognition and machine learning,” *Journal of electronic imaging*, **vol. 16**, no. 4, p. 049901, 2007.
- [46] M. E. Savu, “Segmentarea regiunilor de interes din imagini de retină, folosind rețele neuronale convoluționale,” *Lucrare de Disertație*, Universitatea POLITEHNICA București, 2017.
- [47] A. Ng, „Machine Learning”, Stanford University, 2018 [accessed 10-June-2018] Available [Online]: <https://www.coursera.org/learn/machine-learning/home/welcome>;